ESPRIT — École Supérieure Privée d'Ingénierie et de Technologies

[TO FILL: Department / Engineering track name]

Final Year Engineering Project Report

(Projet de Fin d'Études)

CANvisionNative

A Hybrid-Architecture Intrusion Detection System
for Vehicle CAN Bus Networks

Prepared by: **[TO FILL: Student full name]**

Academic supervisor: **[TO FILL: Academic supervisor name]**

Company supervisor: **[TO FILL: Company supervisor name, or "N/A" if independent project]**

Host organization: **[TO FILL: Host company name, or "Independent project"]**

Academic Year: **[TO FILL: 20XX / 20XX]**

Acknowledgements

[TO FILL: Personalize this page. A short suggested structure is given below — replace with your own words.]

I would like to thank my academic supervisor, **[TO FILL: supervisor name]**, for the guidance and feedback given throughout this project.

[TO FILL: If applicable: I would also like to thank my company supervisor, [name], and the team at [company], for their support and for giving me the opportunity to work on this project.]

I would also like to thank the teaching staff at ESPRIT for the technical background that made this project possible, and my family and friends for their support during this final year.

Abstract

Modern vehicles, and electric vehicles in particular, rely on the Controller Area Network (Controller Area Network (CAN)) bus to let their electronic control units exchange data. The CAN protocol was designed for reliability on a closed bus, not for security: it has no built-in authentication or encryption. This makes it an open target for attacks such as message injection, denial-of-service flooding, and replay attacks.

This report presents **CANvisionNative**, a hybrid-architecture Intrusion Detection System (Intrusion Detection System (IDS)) for vehicle CAN bus networks. The system combines a Windows desktop client, built with Windows Presentation Foundation (WPF) and Model-View-ViewModel (MVVM), with a Python backend exposing a Representational State Transfer (REST) Application Programming Interface (API) built on FastAPI. The backend decodes raw CAN frames using vehicle-specific CAN Database (signal definition file format) (DBC) files, extracts timing and payload features, and scores each frame with a fusion of four detection layers: a timing-anomaly layer, a payload-continuity layer, a per-vehicle Isolation Forest machine-learning layer, and a temporal-persistence layer. Detected anomalies are explained in natural language through an AI explanation pipeline that combines rule-based reasoning with a Large Language Model (Large Language Model (LLM)) provider chain.

The system supports three operating modes: replay of recorded logs (Comma-Separated Values (CSV), Measurement Data Format version 4 (ASAM MDF4) (MF4), candump, Vector CAN), live simulation, and live hardware capture through an ELM327 or SocketCAN adapter. It was trained on more than seven million CAN frames from sixteen vehicle- and dataset-specific sources and validated on a real one-hour Kia EV6 recording containing 97,741 frames.

A significant part of this project consisted of a rigorous validation and correction phase. A full root-cause analysis of the alerts produced on a real recording led to four confirmed software defects, found and fixed: a vehicle-model selection bug that caused every replay to be scored with a generic model instead of the correct vehicle profile; a driving-state misclassification bug that labelled 99.93% of a real driving session as “parking”; a reason-attribution bug that let raw, unbounded diagnostic values dominate the displayed root-cause label instead of the actual fusion-score inputs; and a replay-parser inconsistency that caused some log formats to bypass the canonical detection pipeline. Re-validating these fixes end to end in the live application then surfaced two further defects: an alert-polling timer left permanently stopped after an import, and a stale-data leak in the AI explanation cache. Each of the six fixes was proven with direct, reproducible evidence rather than by tuning thresholds. After the corrections, the alert count on the validation recording dropped from 15,945 to 11,971 (a 25% reduction) while ML-score

saturation dropped from 78% to 0%, with no artificial suppression of alerts.

This report documents the requirements, architecture, implementation, and validation of CANvisionNative, and shows that a desktop-class, explainable, multi-layer IDS can be built and rigorously verified for real vehicle CAN bus data using commodity hardware and open datasets.

Keywords: Controller Area Network, Intrusion Detection System, Electric Vehicle, Machine Learning, Isolation Forest, Anomaly Detection, Large Language Model, Automotive Cybersecurity.

Résumé

Les véhicules modernes, en particulier les véhicules électriques, reposent sur le bus CAN (Controller Area Network) pour permettre à leurs calculateurs (Electronic Control Unit (ECU)) d'échanger des données. Le protocole CAN a été conçu pour la fiabilité sur un bus fermé, et non pour la sécurité : il ne dispose d'aucune authentification ni chiffrement natif. Il constitue donc une cible ouverte pour des attaques telles que l'injection de trames, les attaques par déni de service (Denial of Service (DoS)) et les attaques par rejeu.

Ce rapport présente **CANvisionNative**, un système de détection d'intrusion (IDS) à architecture hybride pour les réseaux CAN de véhicules. Le système combine un client de bureau Windows, développé en WPF selon le patron MVVM, avec un backend Python exposant une API REST construite avec FastAPI. Le backend décode les trames CAN brutes à l'aide de fichiers DBC spécifiques à chaque véhicule, extrait des caractéristiques temporelles et de contenu, puis évalue chaque trame par la fusion de quatre couches de détection : une couche d'anomalie temporelle, une couche de continuité de charge utile, une couche d'apprentissage automatique (Isolation Forest) propre à chaque véhicule, et une couche de persistance temporelle. Les anomalies détectées sont expliquées en langage naturel grâce à un module d'explication combinant un raisonnement à base de règles et une chaîne de modèles de langage (LLM).

Le système prend en charge trois modes de fonctionnement : le rejeu de journaux enregistrés (CSV, MF4, candump, Vector CAN), la simulation en direct, et la capture matérielle en direct via un adaptateur ELM327 ou SocketCAN. Il a été entraîné sur plus de sept millions de trames CAN provenant de seize sources de véhicules et de jeux de données, et validé sur un enregistrement réel d'une heure d'une Kia EV6 contenant 97 741 trames.

Une part importante de ce projet a consisté en une phase rigoureuse de validation et de correction. Une analyse complète des causes racines des alertes produites sur un enregistrement réel a mené à quatre anomalies logicielles confirmées, identifiées et corrigées : un défaut de sélection du modèle véhicule qui faisait que chaque rejeu était noté avec un modèle générique au lieu du profil du véhicule correct ; un défaut de classification de l'état de conduite qui étiquetait 99,93 % d'une session de conduite réelle comme "stationnement" ; un défaut d'attribution de la cause qui laissait des valeurs diagnostiques brutes et non bornées dominer l'étiquette affichée au lieu des véritables contributions du score de fusion ; et une incohérence du parseur de rejeu qui faisait que certains formats de journaux contournaient le pipeline de détection canonique. La revalidation de bout en bout de ces corrections dans l'application réelle a ensuite révélé deux anomalies supplémentaires : un minuteur d'interrogation des alertes resté définitivement arrêté après un import, et une fuite de données obsolètes dans le cache d'explications Artificial Intelligence (AI).

Chacune des six corrections a été prouvée par des preuves directes et reproductibles plutôt que par un simple réglage de seuils. Après ces corrections, le nombre d'alertes sur l'enregistrement de validation est passé de 15 945 à 11 971 (soit une réduction de 25 %), tandis que la saturation du score Machine Learning (ML) est passée de 78 % à 0 %, sans suppression artificielle des alertes.

Ce rapport documente les besoins, l'architecture, l'implémentation et la validation de CANvisionNative, et montre qu'un système IDS multicouche, explicable et de classe bureautique peut être construit et rigoureusement vérifié pour des données réelles de bus CAN de véhicule, en utilisant du matériel courant et des jeux de données ouverts.

Mots-clés : Bus CAN, Système de détection d'intrusion, Véhicule électrique, Apprentissage automatique, Isolation Forest, Détection d'anomalies, Modèle de langage, Cybersécurité automobile.

Contents

List of Figures

List of Tables

List of Abbreviations

AI Artificial Intelligence.

API Application Programming Interface.

CAN Controller Area Network.

CAN FD CAN with Flexible Data-rate.

CSV Comma-Separated Values.

DBC CAN Database (signal definition file format).

DoS Denial of Service.

ECU Electronic Control Unit.

HTTP Hypertext Transfer Protocol.

IDS Intrusion Detection System.

ISO-TP ISO Transport Protocol (ISO 15765-2).

JSON JavaScript Object Notation.

LLM Large Language Model.

MF4 Measurement Data Format version 4 (ASAM MDF4).

ML Machine Learning.

MVVM Model-View-ViewModel.

OBD-II On-Board Diagnostics, second generation.

REST Representational State Transfer.

UDS Unified Diagnostic Services.

WPF Windows Presentation Foundation.

General Introduction

Vehicles have become distributed computer networks on wheels. A modern car, and even more so an electric vehicle, contains dozens of ECUs that control braking, battery management, motor torque, infotainment, and driver-assistance functions. These ECUs talk to each other over shared internal networks, and the most common of these networks, by far, is the CAN bus.

The CAN protocol was standardized in the 1980s for industrial reliability: every node on the bus can send and receive frames, and the protocol guarantees that the highest-priority message wins in case of collision. This design works extremely well for a closed, trusted environment. It was never designed to resist an attacker who gains access to the bus, because it has no field for authentication and no encryption. Once an attacker can inject frames onto the bus, the bus itself cannot tell a legitimate frame from a forged one.

Over the last fifteen years, security researchers have repeatedly shown that this is not a theoretical problem. Remote attacks against production vehicles, published diagnostic exploits, and public intrusion-detection datasets have all demonstrated that a compromised CAN bus can affect physical vehicle behaviour. At the same time, the rise of electric vehicles has increased the number of safety-critical messages on the bus (battery management, motor control) and the number of remote attack surfaces (telematics, over-the-air updates, mobile apps).

An intrusion detection system for the CAN bus has to work under constraints that are different from a typical network IDS: frames are small (up to 8 bytes of payload), periodic, and vehicle-specific, and a single trained model rarely generalizes across manufacturers. This motivates a system that combines several complementary detection strategies (timing, payload, machine learning, temporal persistence) and that adapts to the specific vehicle being monitored.

This report describes the design, implementation, and validation of **CANvisionNative**, a hybrid desktop and backend IDS built to detect anomalies on recorded and live CAN traffic, to explain detected anomalies in plain language, and to export the results for later analysis. The project also includes a full engineering validation phase, in which the system was tested against a real one-hour vehicle recording, defects were identified with concrete evidence, fixed, and re-validated.

Report structure

This report is organized into six chapters.

Chapter 1 presents the context of the project. Chapter 2 reviews the state of the art: the CAN bus and its related protocols, known CAN attacks, existing intrusion detection approaches, and

how machine learning has been applied to this problem in the literature. Chapter 3 analyses the functional and non-functional requirements of the system, its actors and use cases, and the project planning. Chapter 4 presents the overall system architecture: the desktop client, the backend API, the detection pipeline, and the AI explanation engine. Chapter 5 describes the implementation of the main modules. Chapter 6 presents the validation work: the testing methodology, the real-data validation on the Kia EV6 recording, the defects found and fixed, and the before/after measurements. The report ends with a general conclusion, a bibliography, and a set of appendices.

Chapter 1

Host Organization and Project Context

1.1 Introduction

This chapter introduces the organization in which the project was carried out, describes the context that led to the project, states the problem addressed, and lists the objectives that guided the rest of the work. It closes with a short summary that leads into Chapter 2.

1.2 Host organization

1.2.1 Presentation

[TO FILL: Add a short (half-page) presentation of the host company: sector of activity, size, main products or services, location. If this was an independent / self-directed project rather than a company internship, replace this subsection with a short paragraph stating that explicitly, and describe instead the academic framework under which the project was carried out.]

1.2.2 Department

[TO FILL: Name the department, service, or team in which the project took place, and its role inside the organization.]

1.3 Project context

The project was carried out in the broader context of automotive cybersecurity. Modern vehicles, and electric vehicles in particular, are built around internal networks of ECUs that communicate over a CAN bus. This bus was designed for reliability, not for security, and it has been repeatedly shown in the literature to be vulnerable to message injection, flooding, and replay attacks (see Chapter 2).

The project, **CANvisionNative**, is a hybrid-architecture IDS for vehicle CAN bus networks. It combines:

- a Windows desktop client (WPF, MVVM) for visualization, replay control, and analyst interaction;
- a Python backend (FastAPI) that decodes CAN frames, extracts features, scores anomalies, and generates natural-language explanations;
- a machine-learning layer trained on more than seven million CAN frames across sixteen vehicle- and dataset-specific sources;
- support for three operating modes: replay of recorded logs, live simulation, and live hardware capture.

The system was designed to be usable both as a forensic tool — importing a recorded log and reviewing detected anomalies after the fact — and, once hardware is connected, as a live monitoring tool.

1.4 Problem statement

Three problems motivated this project.

First, the CAN bus has no built-in security. Any device connected to the bus can send frames, and no frame carries a sender identity or a signature. An intrusion detection system is therefore one of the only practical ways to add a security layer to an existing vehicle without redesigning its electrical architecture.

Second, a single detection strategy is not enough. Timing-based detection catches flooding attacks but misses slow, low-rate attacks. Payload-based detection catches out-of-range values but misses attacks that stay within plausible ranges. Machine-learning models trained on one vehicle rarely transfer cleanly to another vehicle, because message layout, frequency, and normal-value ranges are vehicle-specific. This suggested a design combining several independent detection layers, fused into a single score, rather than relying on one algorithm.

Third, a raw anomaly score is not enough for a human analyst to act on. A list of alerts with numeric scores does not explain what happened, why the system flagged it, or what to do next. This motivated an explanation layer that turns a detected anomaly into a short, readable description of the likely attack pattern and a recommended action.

Beyond building the system, this project also required a rigorous validation phase, since a detection system that produces incorrect or misleading alerts is not trustworthy. This is documented in full in Chapter 6, which describes six confirmed software defects that were found through evidence-based testing on a real one-hour vehicle recording, and their correction.

1.5 Objectives

The objectives of the project were:

1. Design and implement a multi-layer CAN bus intrusion detection pipeline combining timing, payload, machine-learning, and temporal-persistence signals into a single fusion score.
2. Support multiple input sources: recorded logs (CSV, MF4, candump, Vector CAN), a live attack simulator, and live hardware capture through an ELM327 or SocketCAN adapter.
3. Train and integrate vehicle-specific machine-learning models so that detection adapts to the vehicle being analysed instead of relying on one generic model.
4. Build a desktop client that lets an analyst import a log, replay it, review detected anomalies, and export a report.
5. Add a natural-language explanation layer that describes each anomaly and recommends an action, using a combination of rule-based reasoning and a Large Language Model.
6. Validate the complete pipeline against a real vehicle recording, using measurable, reproducible evidence, and correct any defect found before considering the system ready.

1.6 Conclusion

This chapter presented the organization and context in which CANvisionNative was developed, the problems that motivated it, and the objectives set at the start of the project. The next chapter reviews the state of the art on CAN bus security, existing intrusion detection approaches, and the use of machine learning for this problem, in order to position the choices made in this project.

Chapter 2

State of the Art

2.1 Introduction

This chapter reviews the technical background needed to understand the rest of the report. It covers the CAN bus and the protocols built on top of it, the main attack classes documented against it, existing intrusion detection approaches, and how machine learning has been applied to this specific problem. The chapter ends with a comparison of existing approaches and a statement of the research gap this project addresses.

2.2 Electric vehicles and in-vehicle networks

An electric vehicle replaces the internal combustion drivetrain with a battery pack, one or more electric motors, and a battery management system, but it keeps the same general electrical architecture as a combustion vehicle: dozens of ECUs, each responsible for one function (battery management, motor control, braking, infotainment, driver assistance), connected by one or more internal networks. Electric vehicles typically add safety-critical, high-frequency messages related to battery state, state of charge, and motor torque, which increases the number of messages an intrusion detection system has to understand correctly.

Because manufacturers rarely publish the internal message layout of their vehicles, most of the practical knowledge about a specific vehicle's CAN traffic comes from reverse-engineered DBC files produced by the community, from UDS diagnostic responses, or from vendor tools. This project relies on both sources, as described in Chapter 4.

2.3 The CAN bus

2.3.1 Overview

The Controller Area Network (CAN) is a broadcast serial bus standardized by ISO 11898 [1]. Every ECU on the bus can transmit and receive; there is no central master. Each message (frame) carries an identifier and up to 8 bytes of payload in classic CAN. The identifier does not name a

sender or a receiver — it names the *content* of the message, and every node decides for itself whether a given identifier is relevant to it.

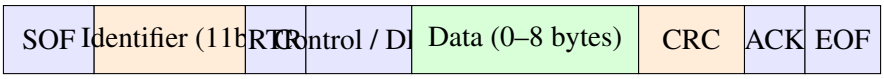

Figure 2.1: Simplified structure of a classic CAN data frame (11-bit identifier).

Figure 2.1 shows the simplified layout of a classic CAN data frame. The identifier also sets the message priority: during arbitration, the frame with the numerically lowest identifier wins the bus, so low identifiers are generally reserved for the most safety-critical messages. This project's replay engine reconstructs exactly these three fields — timestamp, identifier, and payload bytes — from every supported log format, as described in Section 5.4.2.

2.3.2 CAN FD

CAN with Flexible Data-rate (CAN FD) extends classic CAN with two changes: the data payload can go up to 64 bytes instead of 8, and the data phase of the frame can run at a higher bit rate than the arbitration phase. CAN FD is increasingly used on newer vehicle platforms, and one of the raw DBC files used in this project (for the Kia EV6, Section 5.4.4) is itself named for an FD bus. The detection pipeline described in Chapter 4, however, works on a fixed 8-byte payload throughout its feature-engineering and scoring code, matching the classic CAN frames actually present in every recording used for training and validation (Chapter 6).

2.3.3 UDS

Unified Diagnostic Services (UDS), standardized as ISO 14229 [2], defines a request/response diagnostic protocol running on top of CAN. A diagnostic tool sends a service request (for example `0x22 ReadDataByIdentifier` or `0x10 DiagnosticSessionControl`) to a specific ECU address, and the ECU replies with the requested data or an error code. Several datasets used in this project (Chapter 6) were captured by polling a vehicle's ECUs with UDS requests rather than by passively recording broadcast traffic, which changes the expected traffic pattern: request/response pairs at a fixed polling interval, instead of continuously broadcast sensor frames.

2.3.4 ISO-TP

Because a classic CAN frame only carries 8 payload bytes, UDS responses longer than that must be split across several frames. ISO Transport Protocol (ISO 15765-2) (ISO-TP), standardized as ISO 15765-2 [3], defines how this segmentation works: a First Frame announces the total message length, and one or more Consecutive Frames carry the remaining bytes, each with a sequence number. A decoder that only understands single-frame CAN payloads will misinterpret a multi-frame ISO-TP response as several unrelated short frames.

2.3.5 OBD-II

On-Board Diagnostics, second generation (OBD-II) (SAE J1979 [4]) is a standardized subset of vehicle diagnostics, accessible through a physical connector present in virtually every vehicle sold after the mid-2000s. It defines standard parameter identifiers (PIDs) for common values such as vehicle speed or engine coolant temperature. OBD-II adapters, including low-cost ELM327 devices, are one of the most common ways to get CAN bus access to a vehicle without opening the dashboard, and are the entry point supported by this project's live hardware capture mode (Chapter 4).

2.4 Known CAN bus attacks

The CAN bus has no authentication, no encryption, and no reliable way to identify the sender of a frame. This has been demonstrated repeatedly in the security literature. Koscher et al. [5] showed that an attacker with physical access to a vehicle's internal network could manipulate safety-critical functions by simply injecting crafted CAN frames. Miller and Valasek [6] later demonstrated a remote version of this attack, controlling a production vehicle's steering and braking over its cellular connection without any physical access at the time of the attack. These two results are widely cited as the moment automotive cybersecurity became a mainstream concern rather than a theoretical one.

From an intrusion-detection point of view, published attacks and public datasets group into a small number of recurring patterns, all of which this project's detection pipeline is designed to catch:

- **Denial-of-service / flooding.** The attacker transmits a high-priority identifier (often `0x000`) at a very high rate, starving the bus and preventing legitimate frames from being transmitted.
- **Fuzzing.** The attacker transmits frames with random or semi-random identifiers and payloads, looking for an ECU that reacts unexpectedly.
- **Spoofing / injection.** The attacker transmits frames with a legitimate identifier but a forged payload, for example to report a false speed or gear state.
- **Replay.** The attacker records legitimate traffic and re-transmits it later, out of its original context, to re-trigger a past command.
- **Masquerade.** A refinement of injection in which the attacker first suppresses the legitimate sender of an identifier, then injects forged frames on the same identifier at the expected rate, so that simple frequency-based detectors do not notice anything unusual. This pattern is explicitly represented in the ROAD dataset [7], which distinguishes plain injection attacks from their masqueraded counterparts.

Table 2.1 maps each of these patterns to the detection signal it primarily disturbs. No single signal covers every row, which is the main argument, developed in Chapter 4, for combining several of them into one fusion score rather than relying on any one of them alone.

Table 2.1: CAN bus attack patterns and the detection signal each one primarily disturbs.

Attack pattern	**Typical effect on the bus**	**Primary detection signal**
DoS / flooding	Sustained, abnormally high message frequency on one identifier	Timing
Fuzzing	Unexpected identifiers or out-of-range payload bytes	Payload
Spoofing / injection	Legitimate identifier, implausible or out-of-pattern payload	Payload / ML
Replay	Legitimate frames repeated out of their original context	Temporal persistence
Masquerade	Forged frames at the expected rate — timing alone stays quiet	ML

2.5 Intrusion detection systems for CAN

Intrusion detection approaches for the CAN bus are usually grouped into three families.

Signature-based detection matches observed traffic against a database of known attack patterns. It is precise for known attacks but cannot detect a new attack it has not seen before, and CAN traffic rarely carries enough structure (no protocol fields beyond identifier and payload) to write rich signatures.

Specification-based detection encodes expected behaviour by hand: expected identifiers, expected frequency ranges, expected payload ranges. It is transparent and does not need attack data to train, but it needs a specification for every vehicle and message, and it does not generalize to unexpected but legitimate variations in traffic.

Anomaly-based detection learns what normal traffic looks like (from timing, payload statistics, or both) and flags deviations. This is the approach with the widest applicability across vehicles, because it does not need an attack signature or a hand-written specification, only a sample of normal traffic. It is also the family this project belongs to, combined with a persistence layer to reduce single-frame false positives.

2.6 Machine learning for CAN intrusion detection

Because CAN frames are small, frequent, and mostly periodic, unsupervised anomaly detection is a natural fit: normal traffic is abundant, while labelled attack traffic is comparatively rare and vehicle-specific. Two techniques are particularly relevant to this project.

Isolation Forest [8] builds an ensemble of random decision trees that isolate each data point by repeated random splits. Points that are easy to isolate (short average path length across the trees) are scored as more anomalous than points that need many splits to separate from the rest of the data. It does not require labelled attack data, its per-sample cost is low, and it does not assume a particular data distribution, which makes it a common choice for this problem. It is the model used by this project's per-vehicle scoring layer (Chapter 4).

K-Means clustering [9] partitions data into a fixed number of clusters by minimizing the distance between each point and its cluster centroid. In this project it is used not to detect anomalies directly, but to classify the vehicle's current operating state (parking, idle, driving) from timing and payload features, so that the correct per-state Isolation Forest model can be selected. Chapter 6 documents a defect found in this exact classification step and how it was diagnosed and corrected.

Beyond classical models, GAN-based intrusion detection has also been explored in the literature: Seo et al. [10] proposed GIDS (GAN-based Intrusion Detection System), trained and evaluated on the Car-Hacking dataset, one of the most widely used public benchmarks for CAN IDS research together with the ROAD dataset [7] used in this project's own training corpus (Chapter 6).

2.7 Related work and comparison

Table 2.2 compares the main families of approaches discussed above against the design adopted in CANvisionNative.

Table 2.2: Comparison of CAN IDS approaches.

Approach	**Detects unknown attacks**	**Vehicle-specific**	**Needs attack data**	**Human-readable output**
Signature-based	No	Depends on DB	Yes	No
Specification-based	Partially	Yes (by hand)	No	No
Single statistical (timing or payload only)	Partially	No	No	No
Single-model ML (e.g. one Isolation Forest for all vehicles)	Yes	No	No	No
GAN-based (e.g. GIDS [10])	Yes	Depends on training set	No	No
CANvisionNative (this project)	**Yes**	**Yes (per-vehicle model)**	**No**	**Yes (rule-based + LLM)**

Most published anomaly-based approaches evaluate a single detection signal (usually either timing or a single ML model) against a single dataset, and report a numeric score without producing a human-readable explanation of the anomaly. Vehicle-specific model selection, when discussed at all, is usually treated as future work rather than an implemented feature.

2.8 Research gap

From this review, three gaps stand out that motivated the design choices of this project:

1. Most anomaly-based CAN IDS designs in the literature rely on a single detection signal. Combining independent signals (timing, payload, machine learning, temporal persistence) into one fusion score, so that an attack missed by one layer can still be caught by another, is comparatively rare outside research prototypes.

2. Vehicle-specific behaviour is well documented as a challenge, but concrete systems that select a per-vehicle trained model at run time, rather than a single generic model, are

uncommon in practice — and, as Chapter 6 shows, easy to get wrong silently even when the code appears to support it.

3. Explaining a detected anomaly in plain language, rather than presenting a raw numeric score, is largely absent from the reviewed anomaly-detection literature, which focuses on detection accuracy rather than analyst usability.

CANvisionNative addresses these three gaps directly: a four-layer fusion score (Chapter 4), a per-vehicle model store selected at run time (Chapter 4 and Chapter 5), and a rule-based plus LLM explanation pipeline (Chapter 4).

2.9 Conclusion

This chapter presented the CAN bus and its related protocols, the main documented attack classes, and the three families of intrusion detection approaches, with particular attention to how machine learning has been applied to this problem. The comparison in Table 2.2 positioned this project's four-layer, vehicle-specific, explainable design against existing approaches. The next chapter turns this context into a concrete requirements analysis for CANvisionNative.

Chapter 3

Requirements Analysis

3.1 Introduction

This chapter turns the context and objectives from Chapter 1 into a concrete set of requirements. It lists the functional and non-functional requirements of CANvisionNative, identifies its actors and use cases, presents the corresponding use-case, activity, and sequence diagrams, analyses the main project risks, and presents the planning followed during the project.

3.2 Functional requirements

- **FR1 — Import a recorded log.** The system shall let the user import a CAN log file in CSV, MF4, Vector CAN (`.asc`), candump (`.log`), or PCAN text (`.txt`) format, and convert it to a canonical frame stream (timestamp, identifier, payload).
- **FR2 — Select a vehicle profile.** The system shall let the user select the vehicle that produced the log (or the closest matching profile), so that the correct DBC file and the correct trained model are used.
- **FR3 — Replay the log.** The system shall replay the imported frames through the detection pipeline, in order, at a user-selectable speed (0.5× up to a maximum speed), and report progress.
- **FR4 — Detect anomalies.** For every frame, the system shall compute a fusion anomaly score from four detection layers (timing, payload, machine learning, temporal persistence) and raise an alert when the score exceeds the configured threshold.
- **FR5 — Classify severity.** Each alert shall be labelled with a severity level (LOW, MEDIUM, HIGH, CRITICAL) derived from the fusion score.
- **FR6 — Explain an alert.** The system shall let the user request a natural-language explanation of any alert, including the dominant detection layer, a plain-language description of the likely attack pattern, and a recommended action.

- **FR7 — Export results.** The system shall let the user export the alert list as a CSV file and as a formatted PDF report.
- **FR8 — Live simulation.** The system shall be able to generate a simulated live CAN stream, including a simulated attack burst, for testing the detection pipeline without a physical vehicle.
- **FR9 — Live hardware capture.** The system shall be able to connect to a physical CAN adapter (ELM327 or SocketCAN) and feed real vehicle frames into the same detection pipeline used for replay.
- **FR10 — Record a session.** The system shall be able to record a live or simulated session to a log file, optionally injecting a labelled attack, for later replay or retraining.
- **FR11 — Configure the system.** The system shall let the user configure the alert threshold, the LLM provider key, and the hardware interface settings, and persist this configuration between sessions.

3.3 Non-functional requirements

- **NFR1 — Performance.** The replay engine shall process a one-hour, 97,741-frame recording in a few minutes, not hours, on a standard desktop machine. Chapter 6 reports a measured throughput of about 286 frames per second after the corrections described there.
- **NFR2 — Reliability.** A detection defect must be provable and fixable with concrete evidence, not by blindly tuning thresholds. This principle governed the whole validation phase in Chapter 6.
- **NFR3 — Usability.** An analyst without machine-learning expertise shall be able to import a log, read the alert list, and understand the explanation of an alert without consulting external documentation.
- **NFR4 — Portability of input formats.** Every supported log format shall enter the detection pipeline through the same canonical frame representation, so that detection behaviour does not depend on which format was used to record the data.
- **NFR5 — Extensibility.** Adding a new vehicle profile shall only require adding a DBC file and a trained model directory, without changing the detection engine code.
- **NFR6 — Graceful degradation.** If the LLM provider is unavailable, the system shall still produce a rule-based explanation instead of failing the request.
- **NFR7 — Platform.** The desktop client shall run on Windows 10/11 using .NET Framework 4.8 and WPF; the backend shall run as a local Python process exposing a REST API on `localhost`. The full software and hardware requirements are listed in Table B.1 and Table B.2 (Appendix B).

3.4 Actors

- **Analyst (primary user).** Imports logs, starts replays, reviews alerts, requests explanations, exports reports, configures the system.
- **Vehicle / CAN bus (external system).** Source of live frames when the hardware capture mode is used.
- **LLM provider (external system).** Supplies natural-language text for the explanation layer (Gemini, or a local Ollama model as fallback).
- **CANvisionNative backend (system under design).** Performs decoding, feature extraction, scoring, and explanation generation, and exposes them through the REST API.

3.5 Use cases

Figure 3.1 shows the use-case diagram for the Analyst actor, the two external systems, and the main use cases implemented by the system.

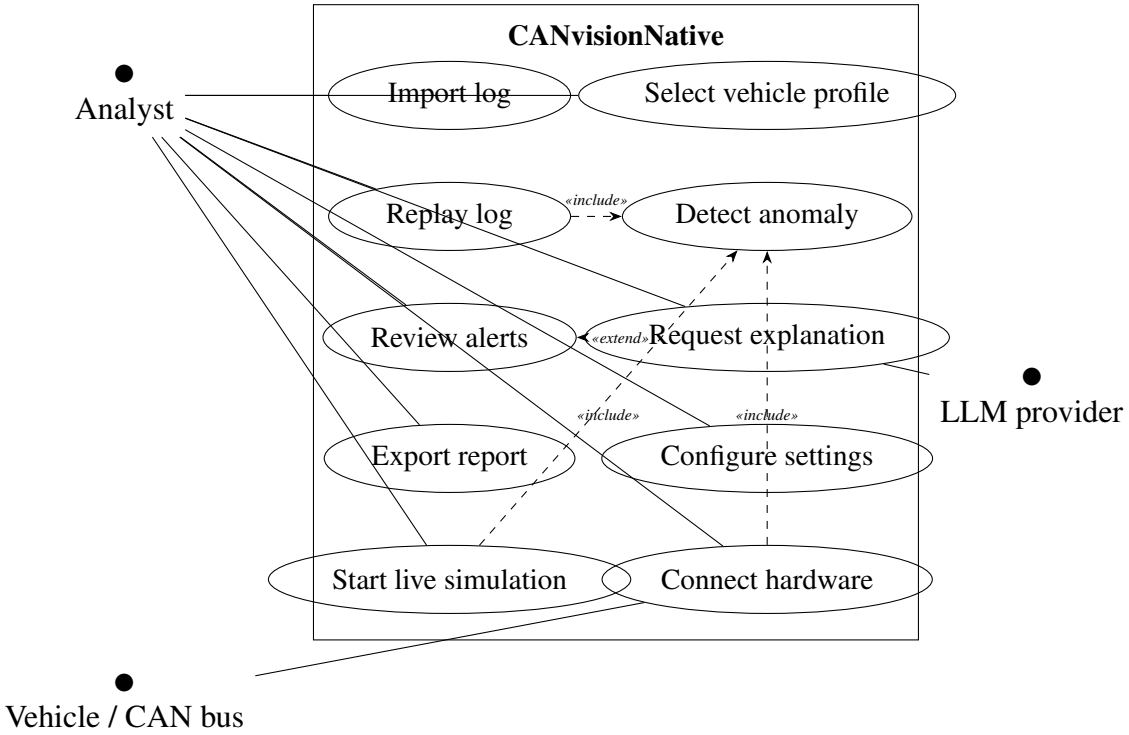

Figure 3.1: Use-case diagram for CANvisionNative.

Table 3.1 details the two most representative use cases.

Table 3.1: Detailed description of the “Replay log” use case.

Field	**Description**
Name	Replay log
Actor	Analyst
Precondition	A log file has been imported and a vehicle profile is selected.
Main flow	1. The analyst clicks *Open File* in the Log Playback view. 2. The client sends the file path and vehicle identifier to the backend. 3. The backend starts a replay subprocess that reads the canonical frame stream. 4. Each frame is scored by the detection pipeline (*Detect anomaly*, included). 5. Alerts are written to a session summary and streamed back to the client.
Postcondition	The alert list, severity distribution, and summary statistics are available for review and export.
Alternate flow	If the replay process exits with an error, the client displays an error state instead of alert results.

3.5.1 Activity diagram – Import and replay

Figure 3.2 shows the activity flow from importing a log to reviewing its alerts.

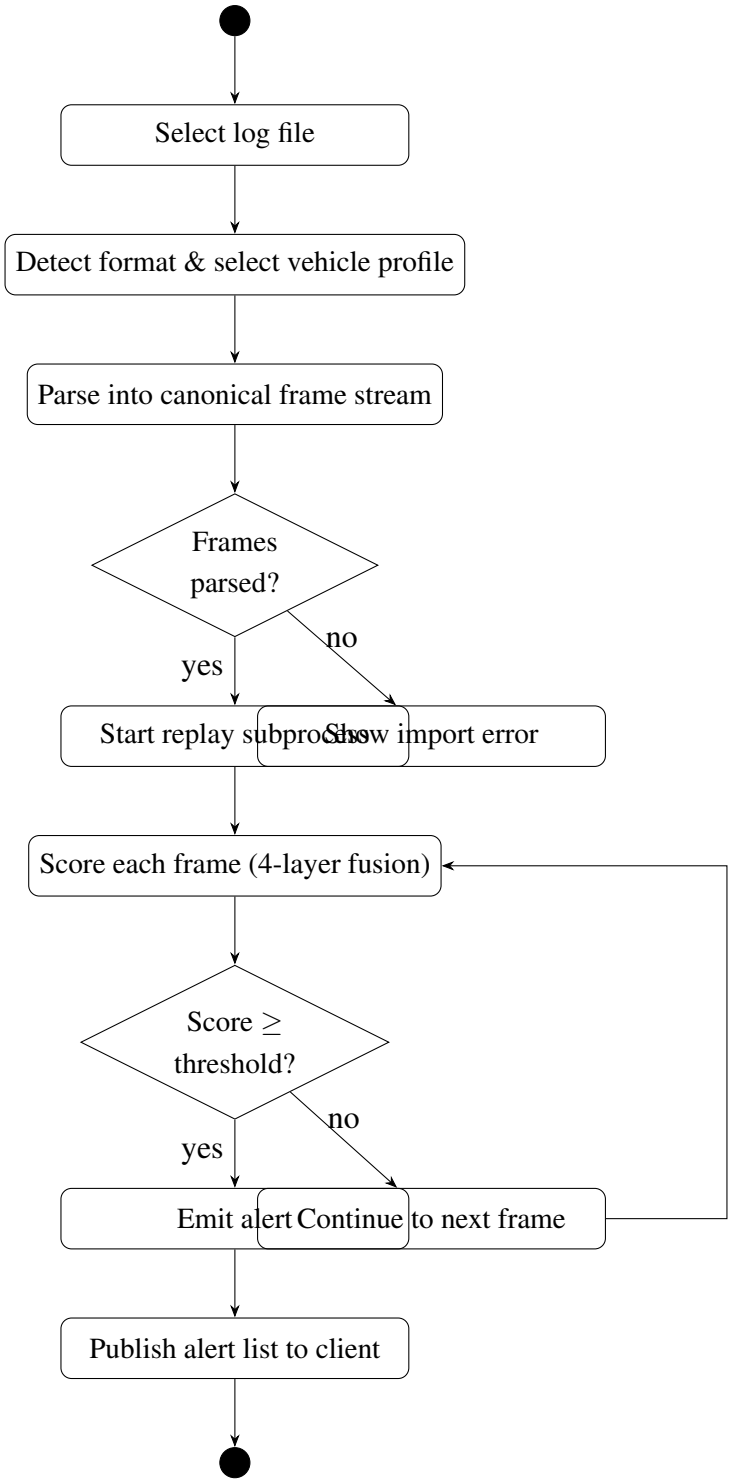

Figure 3.2: Activity diagram: import and replay a log.

3.5.2 Sequence diagram — request an AI explanation

Figure 3.3 shows the sequence of calls triggered when the analyst requests an explanation for an alert, including the fallback behaviour when the primary LLM provider is unavailable.

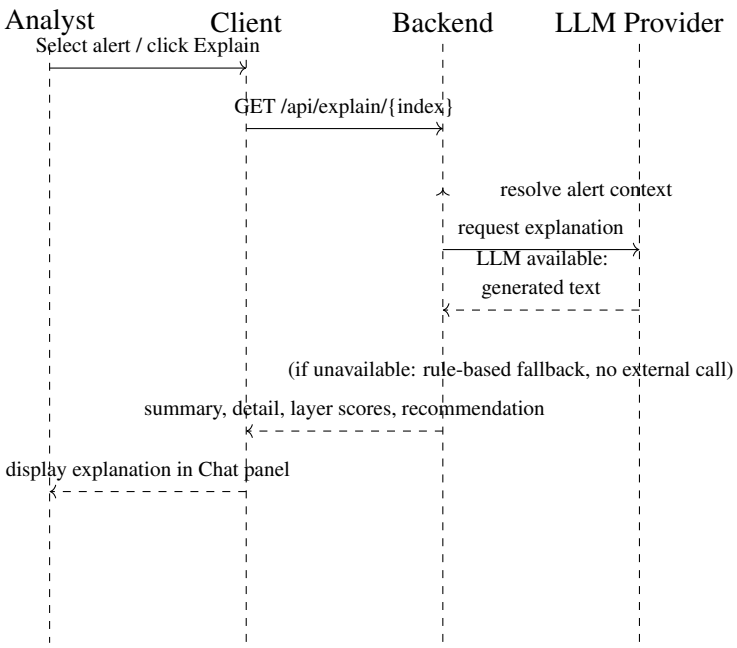

Figure 3.3: Sequence diagram: requesting an AI explanation for an alert.

3.6 Risk analysis

Table 3.2 lists the main risks identified during the project and how each was mitigated.

Table 3.2: Risk analysis.

Risk	**Likelihood**
ELM327 / hardware adapter not available in time	High (materialized)
LLM provider unavailable or unauthenticated	Medium

3.7 Planning

The project was carried out through a sequence of milestones, tracked internally as M1 to M8, followed by a dedicated final validation phase. Figure 3.4 presents this planning as a Gantt chart.

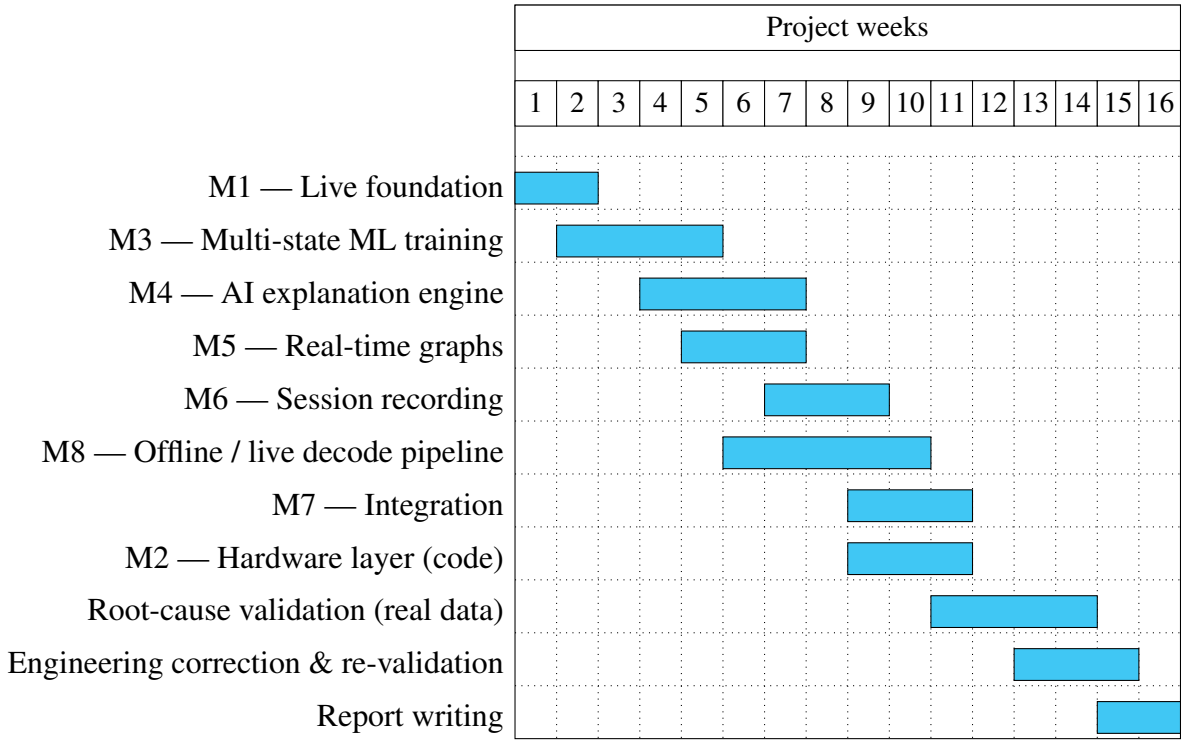

Figure 3.4: Project planning (Gantt chart, in weeks).

Milestone M2 (hardware / OBD2 validation) is shown as code-complete but is explicitly marked as blocked on the physical arrival of an ELM327 adapter, and is not part of the validated scope in Chapter 6, which is why it is not part of the risk-mitigated critical path.

3.8 Conclusion

This chapter defined the functional and non-functional requirements of CANvisionNative, its actors and use cases, and presented the corresponding use-case, activity, and sequence diagrams. It also analysed the main project risks and presented the planning followed. The next chapter presents the system architecture built to satisfy these requirements.

Chapter 4

System Design

4.1 Introduction

This chapter presents the architecture of CANvisionNative: the overall layered architecture, the desktop client, the backend, the database, the REST API, the replay engine, the hardware layer, the detection pipeline, and the AI explanation engine. It closes with the class and entity-relationship diagrams used to implement the design described here.

4.2 Overall architecture

CANvisionNative is built as two cooperating processes on the same machine: a Windows desktop client and a local Python backend, communicating over Hypertext Transfer Protocol (HTTP) on `localhost`. Figure 4.1 shows the layered view of the system.

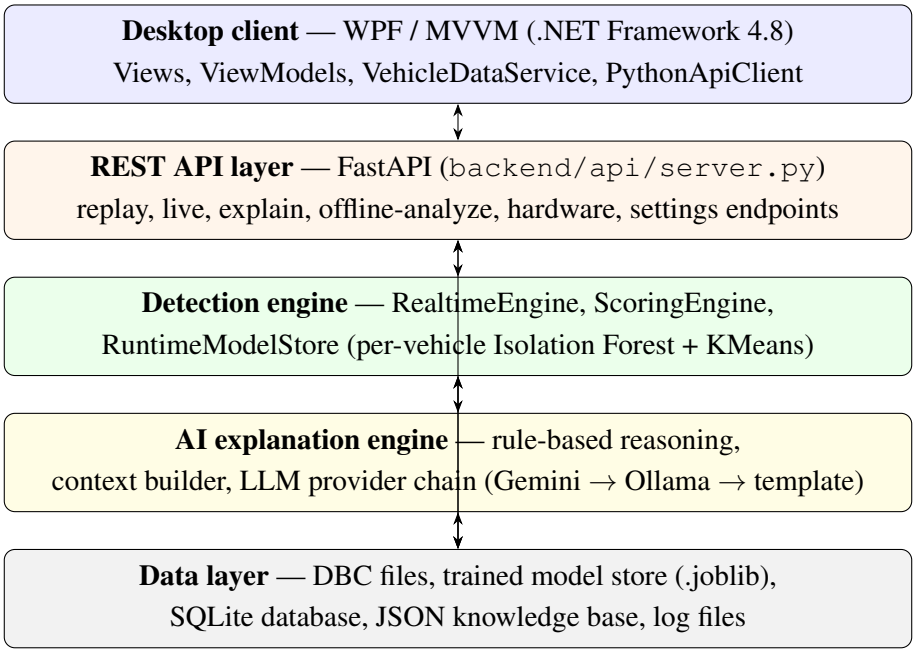

Figure 4.1: Overall layered architecture of CANvisionNative.

4.2.1 Frontend architecture

The desktop client follows the MVVM pattern. Each screen (Home, Dashboard, Telemetry, Diagnostics, Log Playback, Anomaly Intel, Settings) has a XAML view and a corresponding ViewModel in `ViewModels/SectionViewModels.cs`. Two services sit between the ViewModels and the backend, as shown in Figure 4.2:

- **PythonApiClient** — a thin HTTP client wrapping every backend endpoint (start/stop replay, get alerts, get explanation, get vehicle list, and so on).
- **VehicleDataService** — the shared, in-memory state of the client: current snapshot, alert history, replay runtime metrics. It owns a polling timer that periodically calls the backend (replay status, partial alerts, live signals) and raises events that the ViewModels subscribe to, so that every view reflects the same underlying state.

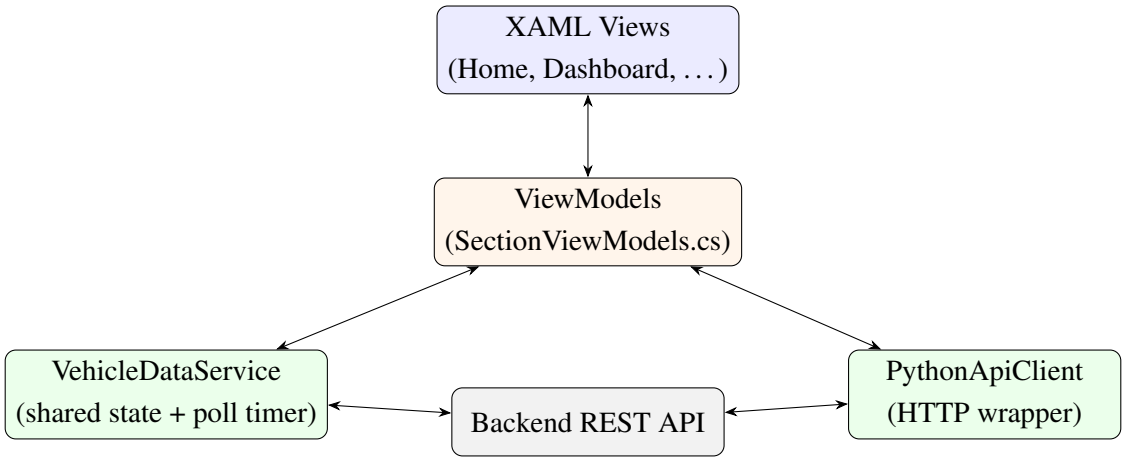

Figure 4.2: Frontend architecture: MVVM views bound to ViewModels, backed by two shared services.

Navigation between sections is driven by a section catalog and a navigation service, so that adding a new screen does not require changes to existing screens.

4.2.2 Backend architecture

The backend is a single FastAPI application (`backend/api/server.py`) organized around five responsibilities:

1. **Decoding** (`backend/decoding/`) — vehicle profile registry and DBC-based signal decoding.
2. **Data processing** (`backend/data_processing/`) — log parsing and MF4-to-CSV conversion.
3. **Machine learning** (`backend/ml/`) — feature engineering, the runtime detection engine, training scripts, and the AI explainer.

4. **Hardware** (`backend/hardware/`) — the live CAN adapter interface and session recorder.

5. **Persistence** (`backend/db/`) — the SQLite session and alert store.

Long-running work (a replay, an offline analysis) is executed in a background subprocess or thread so that the API stays responsive; the client polls a status endpoint until the work completes.

4.2.3 Database

The backend keeps a small SQLite database (`backend/db/canvision.db`) for session bookkeeping and historical alerts, independent of the per-replay summary files used for the live client views. Figure 4.3 shows its three tables.

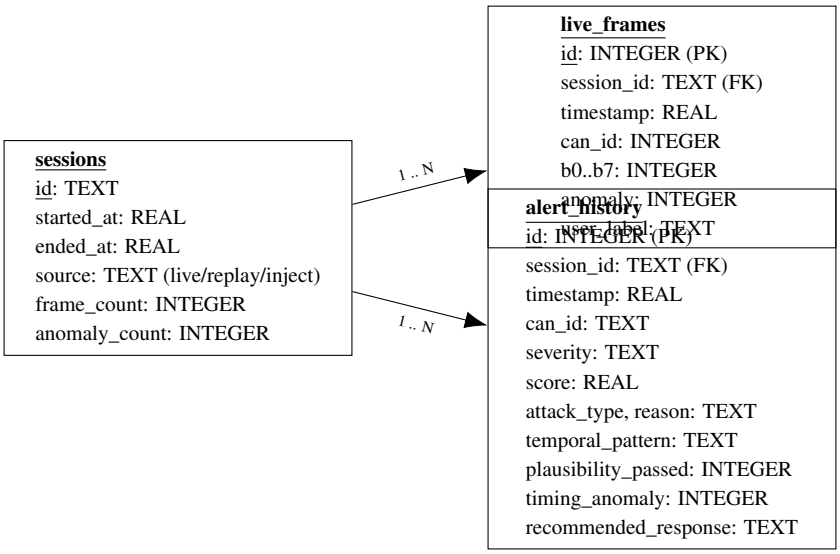

Figure 4.3: Entity-relationship diagram of the SQLite persistence layer.

A `session` groups the frames and alerts produced by one live, replay, or injection run. `live_frames` keeps every captured frame with its raw bytes and the engine's normal/anomalous classification, including an optional `user_label` so a human reviewer can correct the label for future retraining. `alert_history` keeps one row per raised alert, including the parsed detection-layer reason string and the recommended response, indexed on `session_id` and `severity` for fast filtering.

4.3 REST API

The backend exposes more than 40 endpoints under a small number of prefixes: `/api/replay/*` (start, stop, status, alerts), `/api/live/*` (simulation and hardware capture), `/api/explain/{index}` (AI explanation), `/api/offline/*` (batch analysis), `/api/vehicles` (profile registry),

`/api/session/*` (recording), and `/api/settings`. The full endpoint list is given in Appendix A. All endpoints return JavaScript Object Notation (JSON) and are consumed exclusively by the desktop client over `localhost`.

4.4 Replay engine

The replay engine (`backend/ml/runtime/replay_runner.py`) is launched as a subprocess by the `/api/replay/start` endpoint. It reads the input file through a single, format-aware loader (Section 5.4.2) shared with the offline batch analyzer, so that every supported format enters the pipeline as the same canonical `(timestamp, can_id, payload)` stream. Each frame is then passed, in order, to the same `RealtimeEngine` used by the live and simulated paths, guaranteeing that replay, live, and simulated detection behave identically for the same input. Section 5.4.1 describes its implementation in detail, and Chapter 6 documents two defects that were found and fixed in this exact module.

4.5 Detection pipeline

Figure 4.4 shows the per-frame detection pipeline, common to replay, simulation, and live hardware capture.

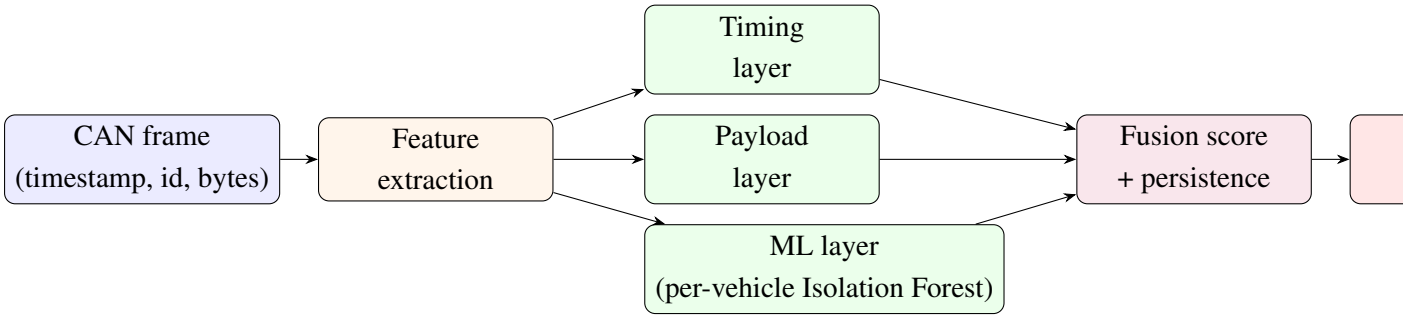

Figure 4.4: Per-frame detection pipeline: feature extraction, four-layer scoring, fusion, and alerting.

Feature extraction computes, per CAN identifier, a rolling set of statistics: inter-arrival time and its variance, message frequency, byte-level difference from the previous frame on the same identifier, and Shannon entropy of the identifier stream and of the payload. These features feed three of the four layers directly:

- The **timing layer** flags both abnormally bursty timing and abnormally high message frequency (a pure timing-variance metric alone would miss a steady flood, since a perfectly periodic flood has almost no timing variance).
- The **payload layer** flags abrupt, out-of-pattern byte changes.
- The **ML layer** first classifies the vehicle's current operating state (parking, idle, driving) with a K-Means model, then scores the feature vector with the Isolation Forest trained

for that state and that specific vehicle profile, falling back to a lightweight heuristic if no matching per-vehicle model is available.

- The **temporal-persistence layer** tracks, over a rolling window, how often the fused score has stayed above the high-severity threshold, so an isolated single-frame spike is treated differently from a sustained anomaly.

The four layers are combined into one fusion score by a fixed weighted sum, and the fused score is compared against four severity thresholds (LOW, MEDIUM, HIGH, CRITICAL). Only frames whose fused score crosses the configured alert threshold are turned into alerts, subject to a per-identifier debounce so that a single burst is not reported as dozens of near-duplicate alerts. The exact weight and threshold values are listed in Table B.3 (Appendix B).

4.6 Hardware layer

The hardware layer (`backend/hardware/can_interface.py`) wraps the `python-can` library to support three transports: ELM327 over a serial/slcan connection, PEAK PCAN-USB, and native SocketCAN. It exposes connect, disconnect, and status endpoints, and feeds every received frame through the same `RealtimeEngine` instance used by replay, with the vehicle identifier selected in the Settings view. This design means the detection pipeline itself required no change to support live hardware once an adapter is available; only the transport layer differs, which is why the code-complete-but-hardware-blocked status described in Chapter 3 was an acceptable risk to carry through the project.

4.7 AI explanation engine

Figure 4.5 shows the explanation workflow triggered when the analyst selects an alert.

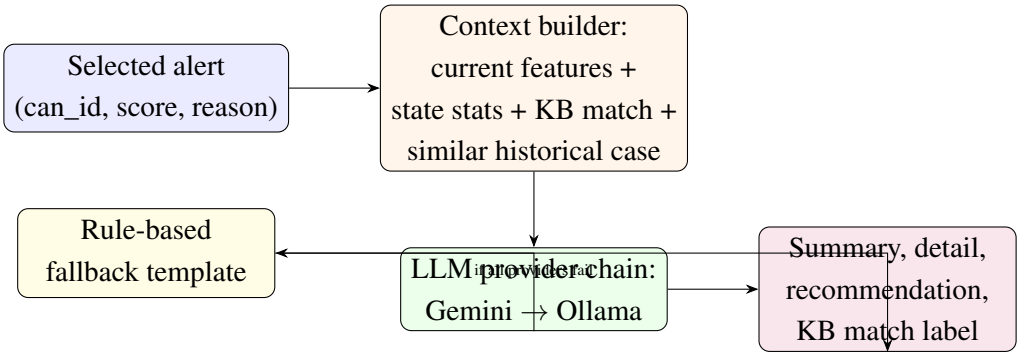

Figure 4.5: AI explanation workflow: context building, LLM provider chain, and rule-based fallback.

The context builder assembles four sources before calling the language model: the current alert's own features (identifier, frequency, entropy, fusion-layer scores), the expected statistics for the vehicle's current state (so the prompt can say, for example, that the observed frequency is

far above the expected range), a keyword match against a small attack knowledge base, and the most similar historical anomaly found by cosine similarity against a persistent, growing store of previously explained anomalies. The assembled context is sent to a language-model provider chain — Gemini first, a local Ollama model second — and, if neither is reachable, a rule-based template still produces a complete, correctly parameterized explanation instead of failing the request, satisfying requirement NFR6.

4.8 Class diagram (backend detection core)

Figure 4.6 shows the simplified class diagram of the backend's detection core, the part of the system most directly involved in the validation work of Chapter 6.

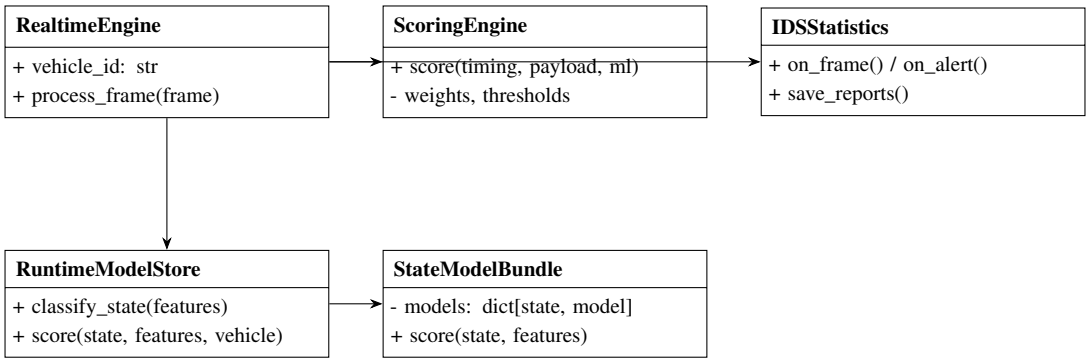

Figure 4.6: Simplified class diagram of the backend detection core.

`RealtimeEngine` is the single entry point used by replay, live simulation, and hardware capture. It owns the current `vehicle_id`, delegates timing/payload scoring and fusion to `ScoringEngine`, and delegates ML scoring to `RuntimeModelStore`, which holds one `StateModelBundle` per vehicle profile (plus a generic fallback bundle) and is responsible for selecting the correct per-state, per-vehicle model. `IDSStatistics` accumulates per-run counters and severity/alert-reason distributions and writes the final summary report consumed by both the desktop client and the AI explanation engine.

4.9 Deployment diagram

Figure 4.7 shows how the system is deployed on a single workstation, with the two external systems it may reach over the network.

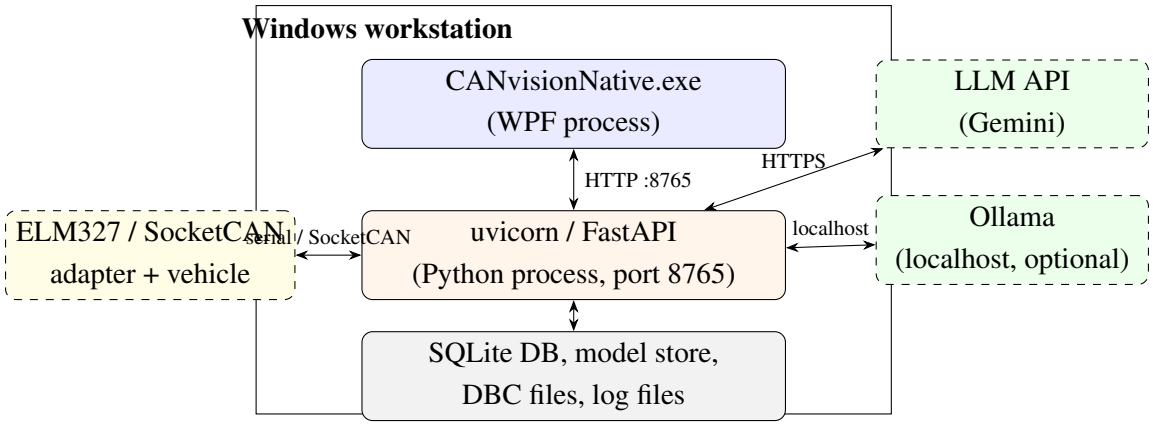

Figure 4.7: Deployment diagram: single-workstation deployment with external LLM and hardware links.

4.10 Conclusion

This chapter presented the layered architecture of CANvisionNative, its desktop and backend components, its database schema, its REST API, the replay engine, the four-layer detection pipeline, the hardware transport layer, and the AI explanation workflow, together with the class, entity-relationship, and deployment diagrams that summarize the design. The next chapter describes how this design was implemented.

Chapter 5

Implementation

5.1 Introduction

This chapter describes how the design presented in Chapter 4 was implemented. It goes through the desktop application, the backend, the replay engine and parsers, the MF4 conversion path, the vehicle profile system, the machine-learning pipeline, the AI explanation module, the charting and export features, the settings module, and the hardware layer. Only short, illustrative snippets are shown; full listings are not included, in line with the reporting guidelines followed for this document.

5.2 Repository organization

The two halves of the system live in the same repository but keep separate top-level folders, reflecting the process boundary between them (Figure 5.1). The C# client's code is split into `UI/` (XAML views), `ViewModels/` (the single `SectionViewModels.cs` file), `Services/` (`PythonApiClient`, `VehicleDataService`, `PdfReportBuilder`), `Models/`, and `Converters/`. The Python backend is split by responsibility under `backend/`: `api/` (the FastAPI entry point), `decoding/` (vehicle profiles, DBC decoding), `data_processing/` (parsers, MF4 conversion), `ml/` (feature engineering, the runtime engine, training scripts, the AI explainer), `hardware/` (the live adapter interface), `db/` (the SQLite store), and `knowledge/` (the attack knowledge base and signal map). Trained models, DBC files, and datasets live under `assets/`, outside both source trees.

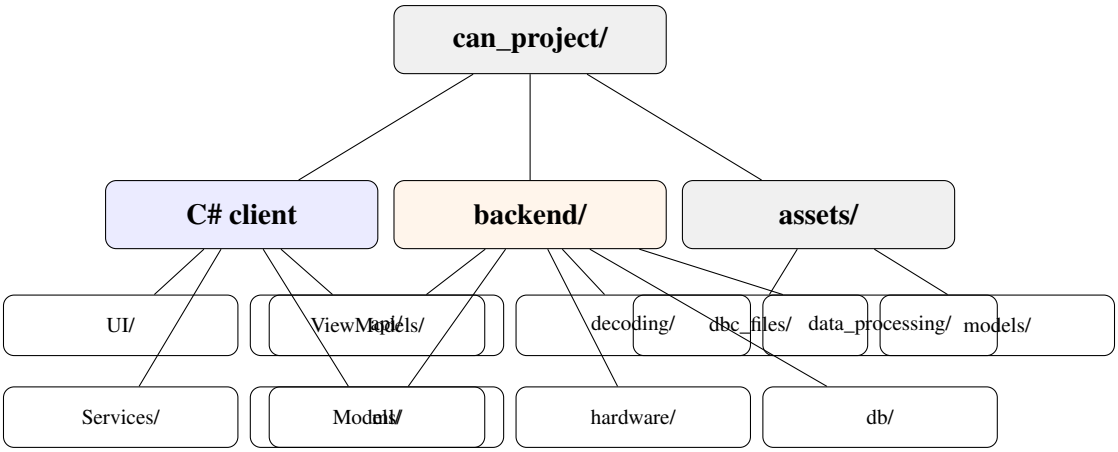

Figure 5.1: Repository organization: the WPF client, the Python backend, and shared assets.

Table 5.1 gives the size of each side of the repository, measured directly on the source tree (excluding generated build output, downloaded datasets, and third-party packages).

Table 5.1: Repository statistics (source code only).

Component	Files	Lines
Python backend (`backend/`)	135	18,363
C# client (`UI/`, `ViewModels/`, `Services/`, `Models/`, `Converters/`)	33	10,084
XAML views (`UI/*.xaml`)	13	—

5.3 WPF application

The desktop client is a WPF application targeting .NET Framework 4.8, structured with the MVVM pattern and CommunityToolkit.Mvvm for command and observable-property boilerplate. The main window hosts a navigation bar (Home, Dashboard, Telemetry, Diagnostics, Log Playback, Settings, Anomaly Intel) backed by a section catalog, so that each section is a self-contained view/ViewModel pair. All ViewModels live in a single file, `ViewModels/SectionViewModels.cs`, an explicit project convention kept throughout development to avoid navigation wiring being duplicated across files.

Two services are shared by every ViewModel:

- `PythonApiClient` wraps every backend call behind a typed method (for example `StartReplayAsync`, `GetAlertExplanationAsync`), using `HttpClient` and Newtonsoft.Json for (de)serialization.
- `VehicleDataService` holds the client-side state shared across views: the current snapshot, the alert history, and the replay/live runtime metrics. It runs a polling loop (`OnRefreshTick`) that periodically asks the backend for replay status, partial alerts, and live signals, and raises events (`AlertHistoryUpdated`, `DataUpdated`) that ViewModels subscribe to.

Figure 5.2 shows the Home screen, the client's entry point: it reports whether the backend is reachable and lets the analyst choose between a live session and an offline import.

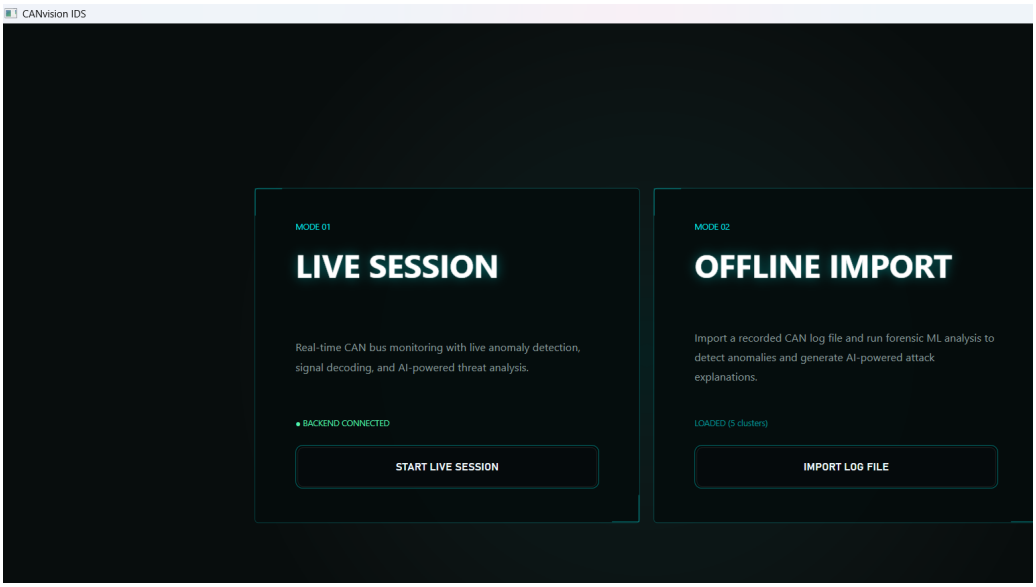

Figure 5.2: Home screen — Live Session / Offline Import choice, backend connection status.

Figure 5.3 shows the Log Playback view during a completed replay of the real Kia EV6 recording used throughout Chapter 6: transport controls, vehicle profile selector, detection-analysis chart, and replay progress panel.

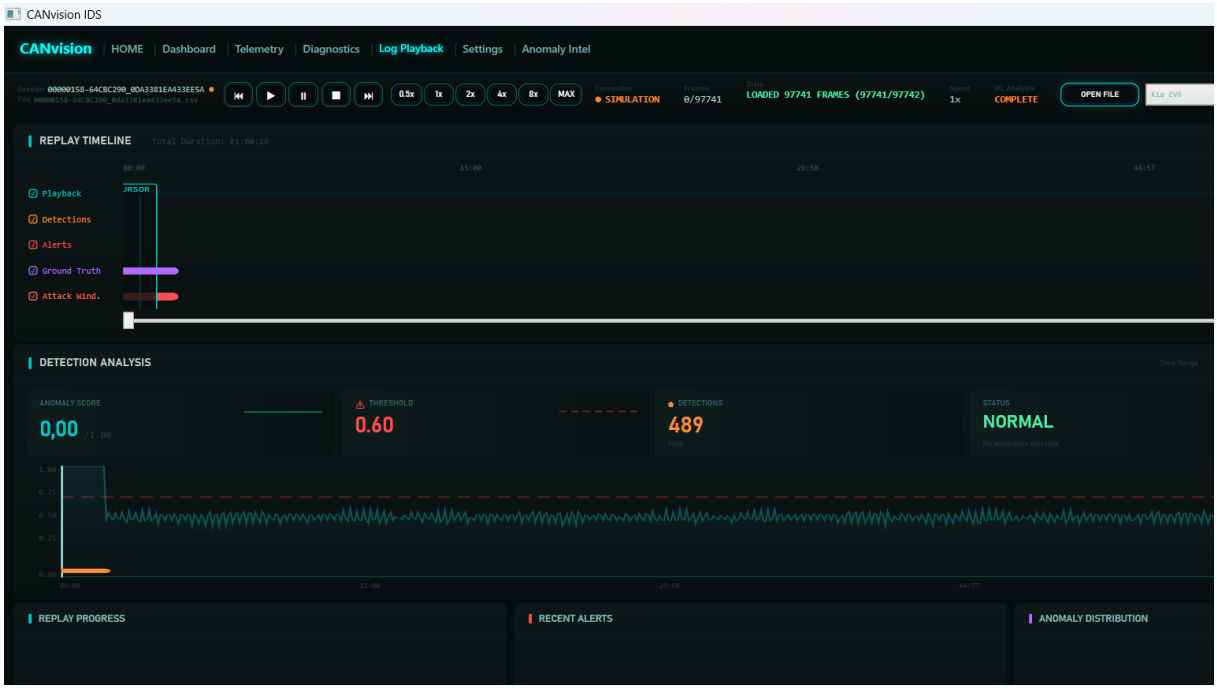

Figure 5.3: Log Playback view after a completed replay of the 97,741-frame Kia EV6 recording.

Figure 5.4 shows the Anomaly Intel view: the alert list on the left, the four-layer attribution bars for the selected alert, and the Chat with AI panel on the right, showing a freshly generated explanation for the selected alert.

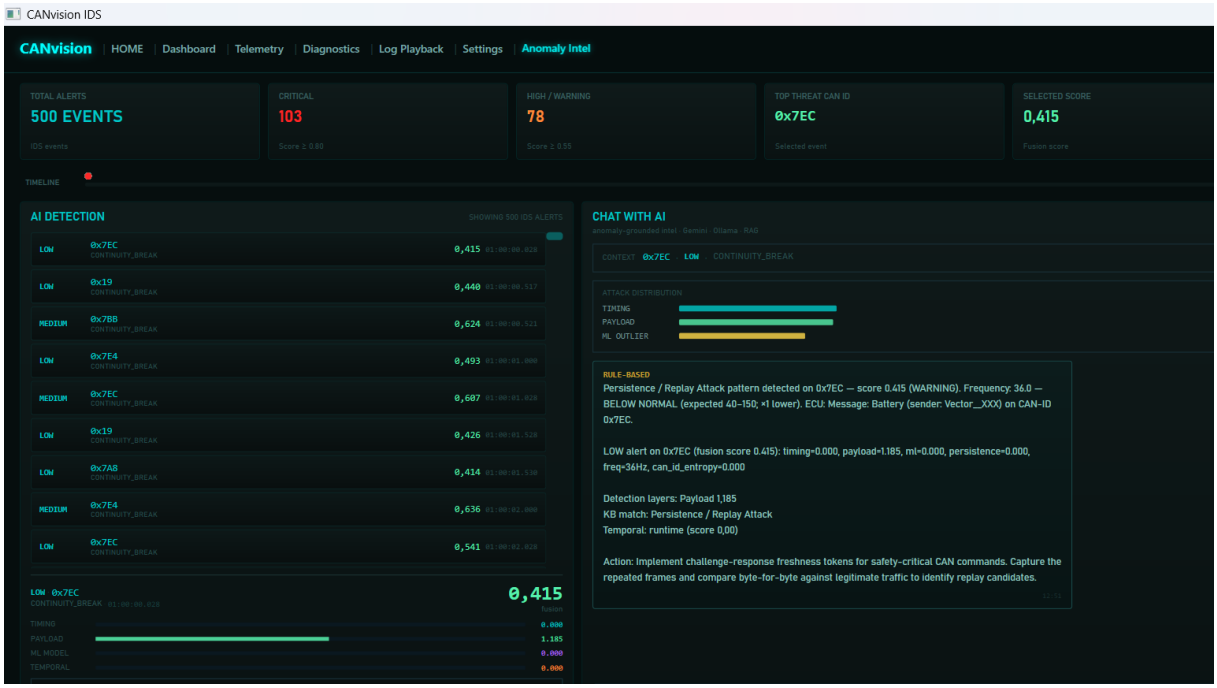

Figure 5.4: Anomaly Intel view: alert list, layer-attribution bars, and the Chat with AI explanation panel.

The remaining two live views, Telemetry and Diagnostics, are shown in Figure 5.5 and Figure 5.6. Telemetry draws the decoded signal channels (speed, state of charge, motor temperature, voltage) alongside the raw live signal monitor and per-identifier activity ranking; Diagnostics exposes adapter and bus health, a running IDS event log, and a set of recovery actions for the live session.

Figure 5.5: Telemetry view: decoded signal channels, live signal monitor, and CAN-ID activity ranking.

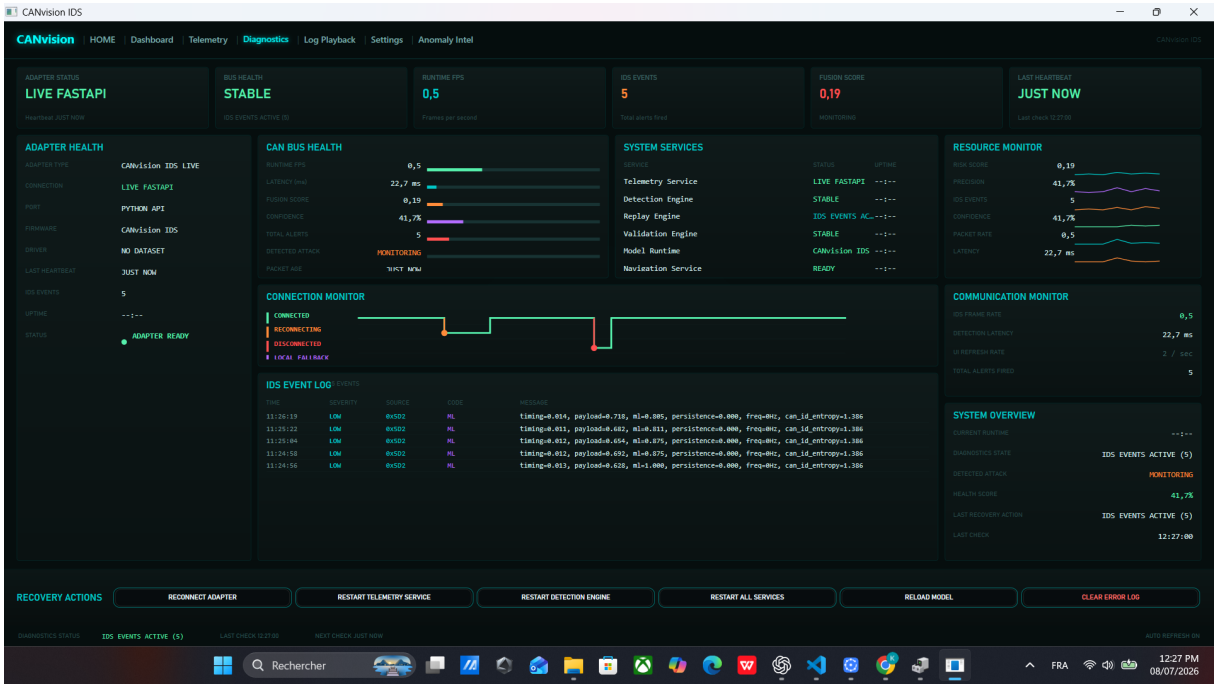

Figure 5.6: Diagnostics view: adapter/bus health, IDS event log, and recovery actions.

5.4 FastAPI backend

The backend entry point, `backend/api/server.py`, defines all REST endpoints in a single FastAPI [11] application. Long-running operations — starting a replay, running an offline batch analysis — are launched as a subprocess or background thread, and the endpoint returns immediately with a status the client can poll. Module-level, process-lifetime state (the current replay process handle, the most recent alert context list, the explanation cache) is protected by explicit locks where it is shared between the request-handling thread and a background monitoring thread.

5.4.1 Replay engine

This section describes the concrete implementation of the replay engine introduced in Section 4.4. `backend/ml/runtime/replay_runner.py` is invoked as a subprocess by `POST /api/replay/start`. It loads the input file through the shared, format-aware loader described in Section 5.4.2, builds three aligned arrays (timestamps, identifiers, payloads), and feeds them one by one into a `RealtimeEngine` instance configured with the requested vehicle identifier. Every 1,000 frames it writes a partial-alerts file to disk so the client can show live progress before the run completes; at the end it writes a summary JSON file containing the full alert list and per-run statistics, which is what `GET /api/replay/alerts` serves to the client.

A background thread on the server side (`monitor_replay_process`) waits for the subprocess to exit, updates the replay status to `completed` or `error`, and — as implemented

during the validation phase in Chapter 6 — publishes this replay's own alerts into the state consumed by the AI explanation endpoint.

5.4.2 Parser and canonical frame representation

Every supported log format is converted, at import time, into the same canonical stream: a sequence of `(timestamp, can_id, payload)` tuples. A single dispatch function, `_load_file`, selects the correct parser by file extension:

- `.csv` — generic `timestamp, can_id, b0..b7` columns, or SavvyCAN-style columns.
- `.mf4` — ASAM MF4 [12] binary logs, decoded with `asammdf` (Section 5.4.3).
- `.asc` — Vector CAN text format.
- `.log` — candump / socketCAN text format, including the timestamped `(seconds) interface id#data` variant used by several public datasets.
- `.txt` — PCAN text format, falling back to candump parsing if the PCAN pattern is not matched.

This single dispatch point is what both the replay engine and the offline batch analyzer call, so that a defect in one format's parser cannot silently produce a different frame stream for replay than for offline analysis. Chapter 6 documents a defect found in exactly this area: before the fix, the replay engine had its own, separate, CSV-only loading code that ignored this shared dispatcher.

5.4.3 MF4 conversion

MF4 files recorded by multi-channel CANedge-style loggers can spread real CAN traffic across several channel groups, most of which are empty template groups. The conversion code (`backend/data_processing/mf4_to_csv_converter.py`) iterates every group in the file explicitly and merges the populated ones, instead of relying on the library's default single-dataframe view, which only surfaces the first group and silently drops the rest. This function is shared between the standalone MF4-to-CSV conversion tool and the format-aware parser described above, so both code paths see the same, complete set of frames.

5.4.4 Vehicle profile system

`backend/decoding/vehicle_profiles.py` defines a small registry of vehicle profiles (Table 5.2), each mapping a vehicle identifier to a bitrate, a UDS-response DBC file (used to decode diagnostic responses), and, where available, a raw broadcast DBC file (used to decode spontaneous, non-diagnostic frames).

Table 5.2: Supported vehicle profiles.

Identifier	Display name	Bitrate
`nissan_leaf`	Nissan Leaf (2019, 40/62 kWh)	500 kbps
`hyundai_ioniq5`	Hyundai Ioniq 5	500 kbps
`hyundai_kia`	Hyundai / Kia (generic)	500 kbps
`skoda_enyaq`	Škoda Enyaq iV (VW MEB)	500 kbps
`vw_skoda_audi`	VW / Škoda / Audi (generic)	500 kbps
`renault_zoe`	Renault Zoe	500 kbps
`tesla_model3`	Tesla Model 3	500 kbps
`kia_ev6`	Kia EV6	500 kbps
`bmw_i3`	BMW i3 (2017, 60 Ah REX)	500 kbps

Selecting a profile in the client sets the `vehicle_id` sent to every replay and live-capture request, which in turn selects both the DBC decoding tables and the trained model bundle used for scoring, as described next. The nine profiles in Table 5.2 are the ones with full signal decoding through a named DBC file; the trained model store (Appendix C) additionally holds model bundles for training-only sources without commercial vehicle names, such as the ROAD and Car-Hacking benchmark datasets, which is why the model store has more entries than Table 5.2.

5.5 Machine learning pipeline

5.5.1 Feature engineering

For every CAN identifier, the engine maintains a small rolling window of recent inter-arrival times and byte-level differences. From this window it derives fourteen features at each frame: the current and rolling-mean/variance inter-arrival time, the current and rolling byte difference, message frequency and rate, a burstiness ratio, the coefficient of variation of inter-arrival time, changed-byte count, and the Shannon entropy of both the identifier stream and the payload bytes. Every feature is clipped to a fixed range and passed through `log1p` before being handed to the models, matching the transform used when the models were trained.

5.5.2 Isolation Forest scoring

Each vehicle profile that has a trained model bundle stores one scikit-learn [13] Isolation Forest [8] per driving state (for example, *idle* and *driving*), each with its own fitted `RobustScaler`. Scoring a frame means: scale the feature vector with that state's scaler, call `decision_function` on the forest, and linearly remap the result so that the model's own decision threshold maps to a score of 0.5, and increasingly negative (more anomalous) values map towards 1.0. If no bundle exists for the requested vehicle, the store falls back to a generic bundle, and if even that has no model for the current state, the engine falls back to a lightweight heuristic derived from the timing and payload layers, so that scoring never fails outright.

5.5.3 State classification

Before scoring, the engine must decide which driving state the vehicle is currently in, since a different Isolation Forest is trained per state. This is done with a K-Means [9] model shared across vehicles: the current feature vector is scaled with a dedicated `RobustScaler` and passed to `kmeans.predict`, and the resulting cluster index is mapped to a state name (*parking*, *idle*, *driving*) through a small lookup table produced at training time.

Two implementation details matter here and are directly connected to a defect documented in Chapter 6: the feature vector passed to the state classifier must have the exact same length and order as the one used to fit the K-Means model (fourteen features), and it must be scaled with `RobustScaler.transform` before calling `predict`, exactly as it was scaled before `fit` during training. Skipping either step does not raise a visible error if the surrounding code catches exceptions broadly, which is exactly what made this defect hard to notice without deliberate, evidence-based testing.

5.6 AI explanation module

The explanation module (`backend/ml/explainer/`) implements the workflow introduced in Section 4.7, and is called from the `GET /api/explain/{alert_index}` endpoint. It builds a context object from four sources — the alert's own features, expected statistics for the current vehicle state, a keyword match against a small JSON attack knowledge base, and the most similar past anomaly retrieved by cosine similarity from a persistent JSON-lines history file — and sends this context to a provider chain that tries Google Gemini first, then a locally running Ollama model, and finally a rule-based template that fills in the same fields (summary, detail, recommendation) directly from the numeric context, so that the explanation is always produced even with no network access. Each resolved explanation is also appended to the historical anomaly store, so that later, similar anomalies can reference it.

Explanations are cached by a key derived from the alert's own identifier and score, to avoid asking the language model the same question twice for the same alert within a run; Chapter 6 documents how the lifecycle of this cache, not the explanation logic itself, was the source of a validation defect, and how it was corrected.

5.7 Charts

The client draws its charts (anomaly score history, per-identifier activity, severity distribution, live signal channels) with plain WPF `Polyline` elements bound to rolling `PointCollection` buffers updated on each data tick, rather than a third-party charting library, keeping the rendering path simple and dependency-free.

5.8 Export

Two export paths are implemented from the Anomaly Intel view, shown in Figure 5.7: CSV export writes the currently displayed alert list, including the parsed reason, KB match label, and explanation text where available, to a user-chosen file; PDF export (`Services/PdfReportBuilder.cs`, built on PDFsharp) renders a paginated A4 report with a header, summary statistics, and a formatted alert table, suitable for sharing outside the application. Both paths read from the same in-memory alert list the analyst is looking at, so an export always matches what is currently on screen.

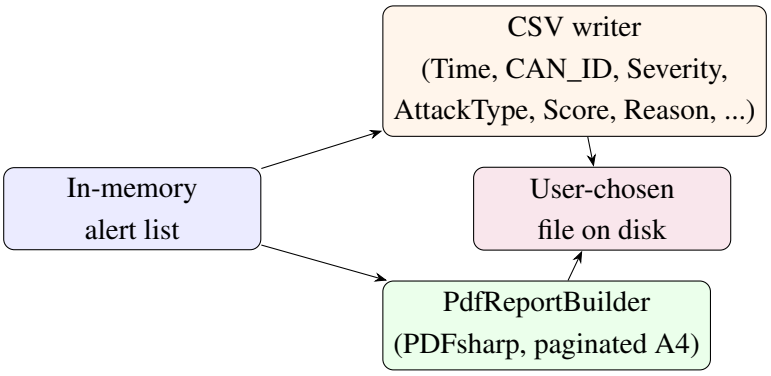

Figure 5.7: Export workflow: both CSV and PDF export read the same displayed alert list.

Appendix D documents one minor, genuine formatting defect found in the CSV path during validation (an unquoted, French-locale decimal comma in the `Score` column) that is left as a known issue rather than corrected here, since the engineering scope of this project was frozen once the six defects of Chapter 6 were fixed and re-validated.

5.9 Settings

The Settings view lets the analyst configure the alert threshold, the LLM provider key (Gemini), and the hardware interface (adapter type, COM port, bit rate). Configuration is persisted to `%AppData%\CANvision\config.json` and reloaded, and any configured LLM key is applied as an environment variable at startup so the backend picks it up without a separate configuration step.

5.10 Hardware

`backend/hardware/can_interface.py` wraps `python-can` to support ELM327 (over an slcan serial connection), PEAK PCAN-USB, and native SocketCAN. Connecting sets the engine's active vehicle identifier so that live frames are scored with the correct per-vehicle model, exactly as a replay does; disconnecting resets it to the generic profile so that a later, unrelated request (for example the synthetic `/vehicle` endpoint used for demo purposes) does not keep scoring against a vehicle that is no longer connected.

5.11 Conclusion

This chapter described the implementation of CANvisionNative's main modules: the WPF client and its two shared services, the FastAPI backend and its replay engine, the shared canonical parser and MF4 conversion path, the vehicle profile system, the machine-learning pipeline (feature engineering, Isolation Forest scoring, and state classification), the AI explanation module, and the charting, export, settings, and hardware layers. The next chapter presents how this implementation was validated against a real vehicle recording, the defects found, and the corrections applied.

Chapter 6

Validation

6.1 Introduction

Building a detection system is only half of the engineering work; the other half is proving that it behaves correctly on real data. This chapter documents the validation phase of CANvisionNative: the testing methodology, an interactive UI testing pass, a full root-cause analysis of the alerts produced on a real vehicle recording, four confirmed software defects found and corrected as a direct result of that analysis, two additional defects found while re-validating the fixes end to end in the live application, and the before/after measurements that quantify the effect of every correction.

6.2 Testing and QA methodology

The validation work followed one rule throughout: *every claim had to be backed by reproducible evidence before any code was changed*. Threshold tuning, blind retraining, or reducing the alert count by construction were explicitly excluded as acceptable fixes. A defect was only considered found once it could be triggered on demand and explained at the code level, and only considered fixed once the same reproduction steps, run again, produced the expected result.

This was applied in three passes, summarized in Table 6.1.

Table 6.1: QA timeline.

Pass	**Description**
1. Interactive UI audit	Manual, jury-style exploratory testing of the running application: every screen, every control, wrong-order actions (importing twice, detecting before a model is loaded, spamming clicks), destructive actions, resizing and minimizing during a replay.
2. Root-cause analysis of alert volume	Full replay of a real one-hour Kia EV6 recording; every one of the resulting alerts explained and attributed to a dominant cause, until coverage reached 100%.
3. Correction and re-validation	Each confirmed software defect fixed individually, re-validated with direct evidence, then a full end-to-end re-run of the corrected pipeline (import, replay, detection, AI explanation, export) on the same real recording.

Figure 6.1 shows the same three passes as a workflow diagram, including the explicit decision point that governed the whole phase: a defect was only fixed once it had been reproduced and explained, never on suspicion alone.

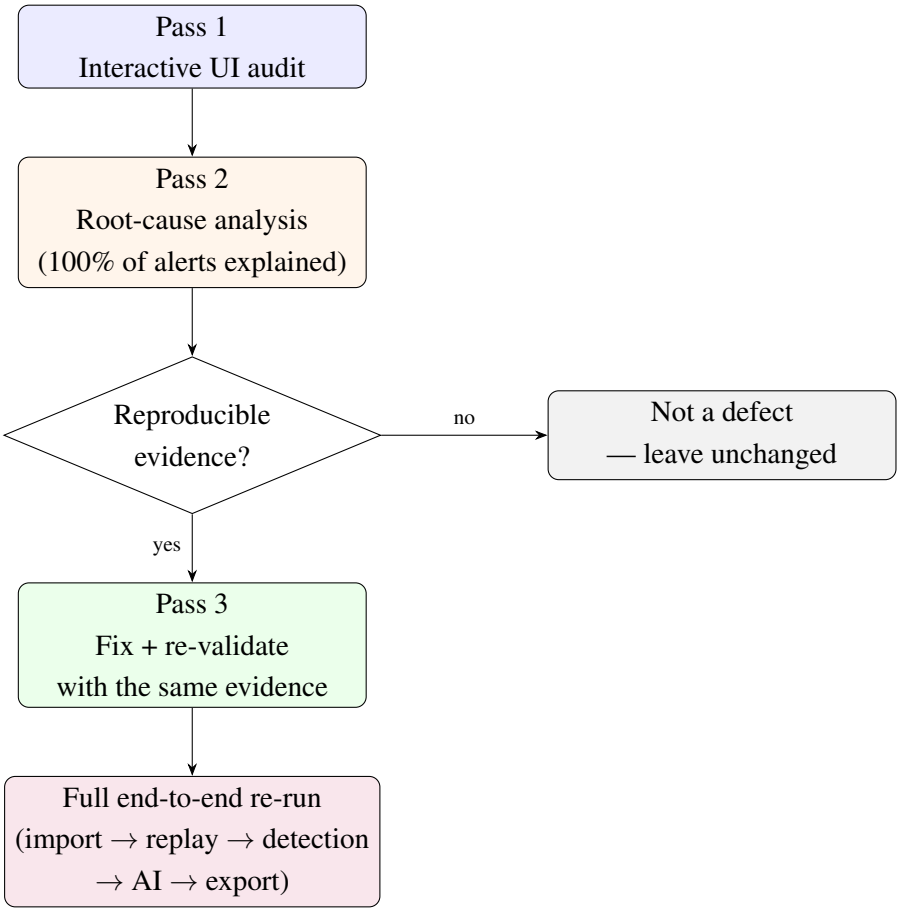

Figure 6.1: Validation workflow: three passes, gated by reproducible evidence.

Table 6.2 lists what was tested at each pass, the method used, and what it covered, to make the scope of the validation phase explicit.

Table 6.2: Testing matrix: what was covered, and how.

Target	Method	Coverage
Desktop UI (all screens)	Interactive UI automation (button clicks, file dialogs, keyboard input)	Navigation, import, replay controls, wrong-order and repeated actions
Detection pipeline	Direct backend API calls + full replay on real data	Vehicle selection, state classification, fusion scoring, reason attribution
Parsers	Synthetic fixtures per format (CSV, candump, Vector CAN) + real recordings	Canonical frame extraction, byte-for-byte comparison against known-good output
AI explanation	Direct API calls across two independent real datasets	Per-alert correctness, cross-session/cross-dataset leakage
Export	Manual export + direct file inspection	CSV column correctness, PDF rendering and pagination
End-to-end	Full application walk-through (import → replay → detection → AI → export)	Integration of every layer above, on the real Kia EV6 recording

6.2.1 Interactive UI audit

Before the data-driven root-cause analysis, the application was exercised interactively, screen by screen, using UI automation to drive the running desktop client (button clicks, file dialogs, keyboard input) while observing the result on screen. This pass covered the Home, Dashboard, Telemetry, and Diagnostics screens, log import with valid, empty, corrupted, and oversized files, the full replay lifecycle (start, pause, resume, stop, replay again), and deliberately wrong-order or repeated actions. It surfaced issues that were fixed before the data-driven validation pass, including an implausible-latency display appearing on three screens, a font glyph-substitution rendering bug, and destructive recovery actions that had no confirmation dialog.

6.2.2 Repository audit

Before changing any code, the repository was audited against its own internal documentation (`ROADMAP.md`, `SETUP.md`, `QA_PLAN.md`) to build an accurate map of every module before touching it, following the same principle applied throughout: understand before modifying, and never fix what is not demonstrably broken.

6.3 Real-data validation setup

6.3.1 Dataset

The main validation dataset is a real, one-hour Kia EV6 recording (`00000158-64CBC290.MF4`) from the CSS Electronics EV Data Pack, captured by a CANedge logger performing active UDS polling of the vehicle's ECUs rather than passively recording broadcast traffic. After MF4 conversion, the recording contains **97,741 frames**. A second, independent dataset — a real Renault Clio DoS-attack candump log, from a different vehicle and a different logger format — was used specifically to test for cross-dataset state leakage (Section 6.9.2).

6.3.2 MF4 validation workflow

Figure 6.2 shows the workflow used to validate the MF4 conversion path before it was trusted for detection validation.

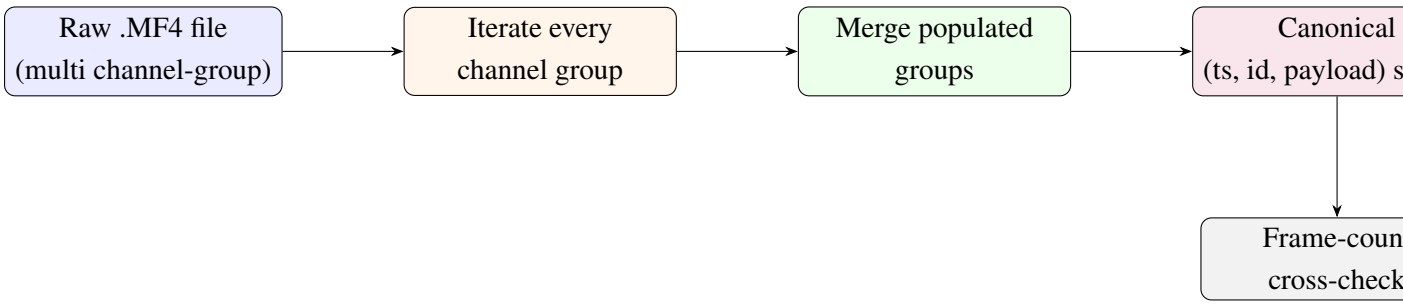

Figure 6.2: MF4 validation workflow: explicit per-group iteration followed by a frame-count cross-check.

The initial, naive conversion (a single call to the library's default dataframe view) only surfaced the first channel group and produced a small fraction of the real traffic. Iterating every group explicitly and cross-checking the resulting frame count against the file's own group metadata confirmed the fix and is what produced the 97,741-frame figure used for every later measurement in this chapter.

6.4 Root-cause analysis of the initial alert volume

Replaying the 97,741-frame recording through the pipeline, before any correction, produced **15,945 alerts** — an alert rate of 16.3%, high enough to question whether the system was usable. Rather than lowering the alert threshold, every alert was categorized by its dominant contributing layer, until 100% of the alerts were explained. Table 6.3 shows the result.

Table 6.3: Root-cause breakdown of the 15,945 baseline alerts.

Dominant cause	Share	Explanation
Machine-learning layer	61.42%	Wrong model selected + state misclassification (Fixes 1 and 2)
Timing layer	31.51%	Sparse UDS polling intervals, structurally ∼50–100% per-identifier alert r
Payload layer	7.07%	Genuine sensor-value entropy in a real battery-telemetry message
Total explained	**100%**	

This breakdown is what directed the correction effort: the ML-dominant majority pointed directly at the model-selection and state-classification defects described next, while the timing- and payload-dominant shares were found to be structurally explainable by the UDS polling pattern of the recording and genuine sensor entropy, and were therefore not treated as defects.

6.5 Fix 1 — Vehicle model selection

Defect. `RealtimeEngine.process_frame()` never passed the selected vehicle identifier to `RuntimeModelStore.score()`, so every replay — regardless of which vehicle profile the analyst selected — was scored with the generic, non-vehicle-specific model.

Fix. The vehicle identifier was threaded through the engine's constructor and every call site that creates it: the replay engine, the offline analyzer, the live hardware interface (including a reset to the generic profile on disconnect), the REST endpoint, and the full C# call chain down to the view model bound to the vehicle selector.

Evidence. The corrected subprocess command line was captured directly from the operating system's process list and confirmed to include `–vehicle kia_ev6`; the `/api/vehicles` endpoint was confirmed to return an identifier (`kia_ev6`) that matches the on-disk trained-model directory exactly, closing the loop from the UI selector to the model actually loaded.

6.6 Fix 2 — Driving-state classification

Defect. The state classifier reported **99.93%** of the recording (97,677 of 97,741 frames) as "parking", despite the recording being an active one-hour drive. Two compounding defects were found in `classify_state()`: the feature vector passed to `kmeans.predict` was built from an 11-feature list intended for the scoring model, not the 14-feature list the K-Means model was actually trained on, which raised `ValueError: X has 11 features, but KMeans is expecting 14 features` on every single call; and this exception was silently caught by a broad `except` clause that always returned "parking", making the failure deterministic rather than intermittent, and therefore invisible without deliberately reproducing the exact exception.

Fix. The feature list used for state classification was corrected to the full 14-feature order the K-Means model was trained on, and the missing `RobustScaler.transform()` call was added before `kmeans.predict()`, matching the training-time scaling exactly.

Evidence. After the fix, the same recording classified as **99.93% "driving"** (97,667 of 97,741 frames), 64 "unknown", and 10 "parking" — the inverse distribution, and a credible one for an hour-long drive.

6.7 Fix 3 — Reason attribution

Defect. The label shown to the analyst as the "dominant detection layer" of an alert was computed by taking the maximum of every `key=value` pair in the alert's internal reason string, including two diagnostic-only fields — `freq` (a raw frequency in Hz, sometimes in the thousands) and `can_id_entropy` (a raw entropy value) — that are never part of the actual fusion score. Because these two fields are numerically far larger than the properly normalized 0–2 range of the four real fusion inputs, they almost always won the comparison, mislabeling the displayed cause of the alert. The same defect, written independently, was found duplicated across four files (six occurrences in total): three in the main API module, and one each in the live hardware interface, the live simulator, and the statistics module.

Fix. All six occurrences were corrected to restrict the comparison to the four fields that genuinely feed the fusion score (timing, payload, machine learning, persistence).

Evidence. `ScoringEngine.score()` was confirmed, by inspecting its signature, to accept only the three normalized layer scores as input (persistence is derived internally from a rolling window), proving the mislabeled field never affected real detection, only the displayed explanation. After the fix, the live `/api/replay/alerts` endpoint was confirmed to return correctly attributed labels (for example `continuity_break` when the payload layer genuinely dominates).

6.8 Fix 4 — Replay parser consistency

Defect. The replay engine had its own, separate loading code that called a generic CSV reader directly, bypassing the shared, format-aware parser described in Section 5.4.2. Any `.log`, `.asc`, or `.txt` file selected through the same file picker used for CSV files — a picker that explicitly allows all of these extensions — was silently mis-parsed as malformed CSV.

Fix. The replay engine was changed to call the same shared, format-aware loader already used by the offline batch analyzer, removing the duplicated CSV-only code path entirely.

Evidence. The corrected CSV path was confirmed to produce byte-identical output to the previous code on all 97,741 frames; synthetic candump and Vector CAN fixtures, which previously produced all-zero timestamps and identifiers, were confirmed to parse their identifier, timestamp, and payload correctly after the fix.

6.9 Additional defects found during end-to-end re-validation

Re-running the corrected pipeline through the live desktop client, rather than through the backend API alone, surfaced two further defects that a purely backend-level test would not have caught.

6.9.1 Live alert-polling stopped permanently

Importing a log from the Home screen calls a service method that stops the client's background polling timer before running its own analysis, and no code path restarted it afterwards. Once this happened, no later replay — even a fresh, correctly configured one — could deliver alerts to the Anomaly Intel view for the rest of the application's lifetime, because the timer that fetches replay results was never running again. The fix guarantees the polling timer is (re)started every time a replay is triggered, regardless of which screen initiated it.

6.9.2 AI explanation cache staleness

Defect. The AI explanation endpoint resolves an alert index against a shared, process-lifetime list that, before this fix, was only ever populated by two other endpoints (offline batch analysis and attack injection) — never by a completed replay. This meant an explanation request for the alert the client was actually displaying could silently resolve against unrelated data left over from a previous analysis or injection run, or, during the short window before a replay finished, against whatever an even earlier session had left behind.

Fix. Two changes closed this: on replay completion, the replay's own alerts are now published into the same shared state already used by the other two endpoints (mirroring a pattern one of them already used); and on every replay *start*, that shared state is cleared immediately, so no previous session's data can be resolved even during the short window before the new replay completes.

Evidence. Two datasets from different vehicles (the Kia EV6 recording and the independent Renault Clio recording, Section 6.3) were replayed one after the other, and the explanation for every checked alert index was confirmed to match that alert's own identifier and score in both datasets, with no trace of the other dataset's data. A second, more targeted test seeded the shared state with the Renault Clio data, started the Kia EV6 replay, and queried the explanation endpoint *while the replay was still running, before completion*: it correctly returned an empty result instead of the seeded stale data, confirming the start-of-replay clearing closes the exact timing window the completion-only fix could not.

6.10 Performance and before/after comparison

Table 6.4 summarizes every measurement taken on the same 97,741-frame Kia EV6 recording, before any correction and after all four data-driven fixes.

Table 6.4: Before/after comparison on the real Kia EV6 recording (97,741 frames).

Metric	**Before**	**After**
Total alerts	15,945 (16.3%)	11,971 (12.25%)
Replay duration	685 s	341.2 s
Throughput	∼143 fps	∼286 fps
`vehicle_state` distribution	99.93% parking	99.93% driving
ML-score saturation ($\geq$0.999)	77.6–78.6%	0.0%

Figure 6.3, Figure 6.4, and Figure 6.5 present the three headline measurements as bar charts, generated directly from the replay summary files produced by the corrected pipeline and from the measurements recorded during the correction phase.

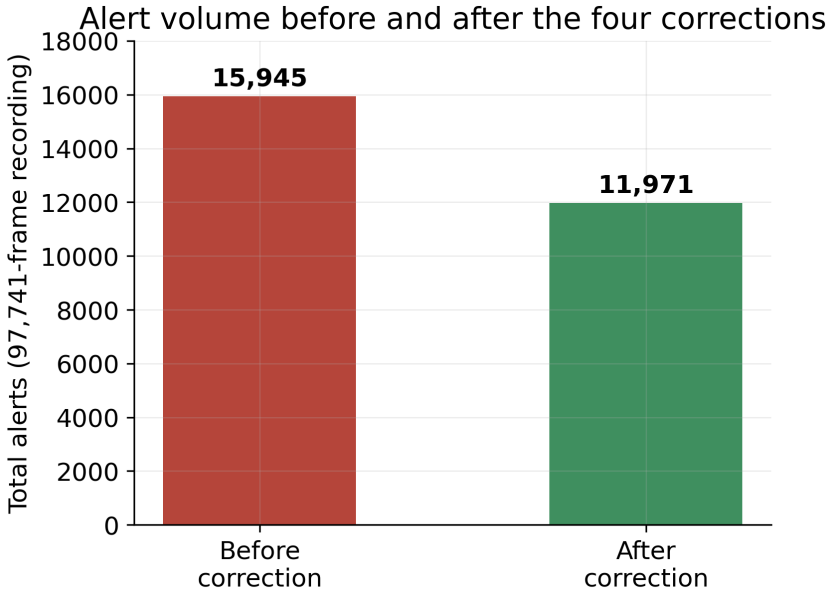

Figure 6.3: Total alerts on the Kia EV6 recording, before and after correction.

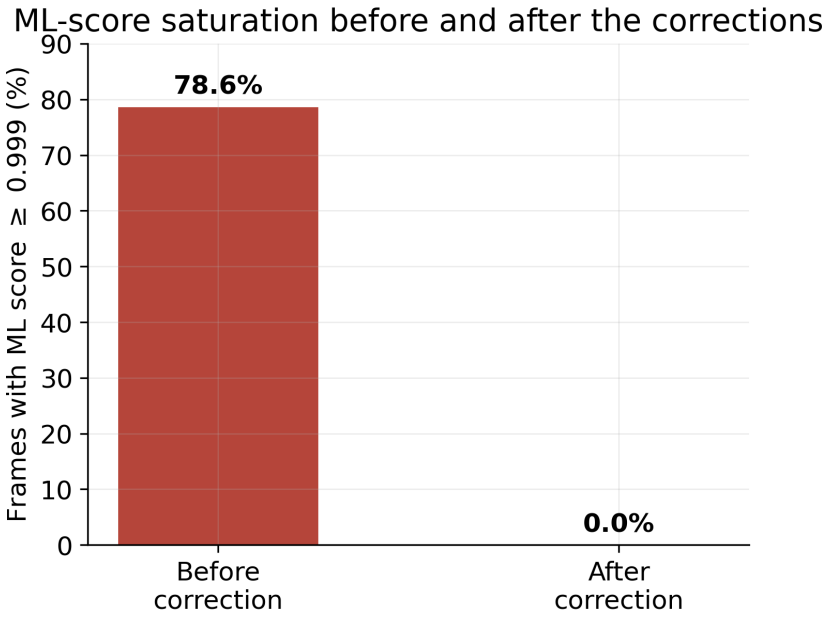

Figure 6.4: Share of frames with a saturated (≥0.999) ML score, before and after correction.

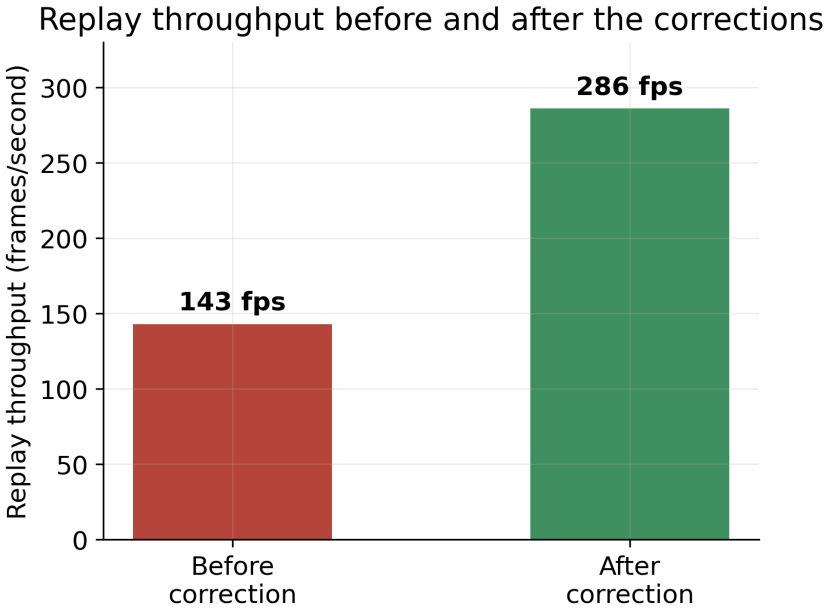

Figure 6.5: Replay throughput, before and after correction.

The alert reduction and the throughput improvement are both side effects of fixing genuine defects (the correct, lighter-weight per-vehicle model being exercised instead of a saturated fallback path), not of loosening the detection threshold, which was left unchanged throughout.

Figure 6.6 isolates the Fix 2 evidence already summarized in Section 6.6: the same 97,741 frames, classified by the state classifier before and after the correction, using the same three state labels on both sides so the two distributions can be compared directly.

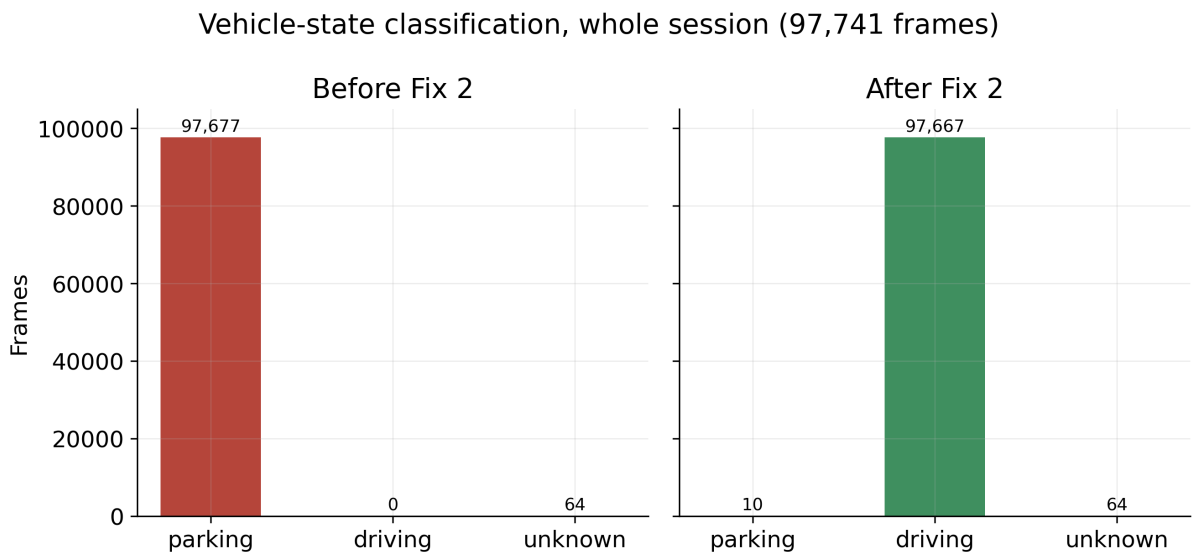

Figure 6.6: Vehicle-state classification of the whole recording, before and after Fix 2.

Figure 6.7 and Figure 6.8 break down the 11,971 post-correction alerts by severity and by dominant detection layer. The reason-distribution chart is itself evidence for Fix 3: none of the three buckets is `can_id_entropy` or `freq`, because those two fields were removed from the comparison entirely and were never part of the real fusion score in the first place (Section 4.5).

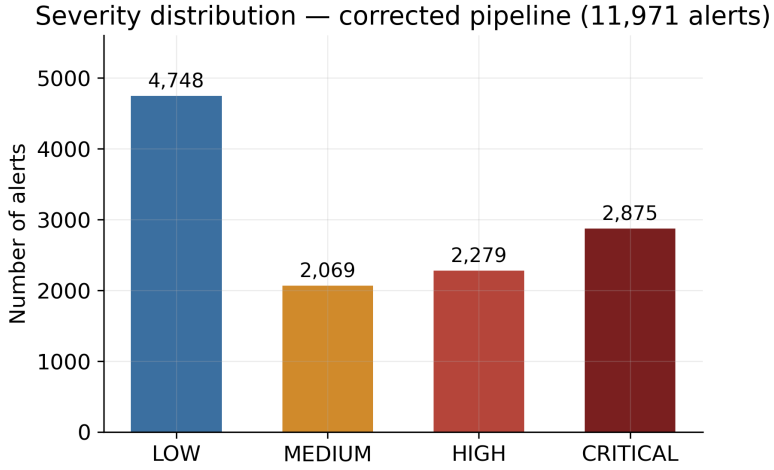

Figure 6.7: Severity distribution of the 11,971 alerts produced by the corrected pipeline.

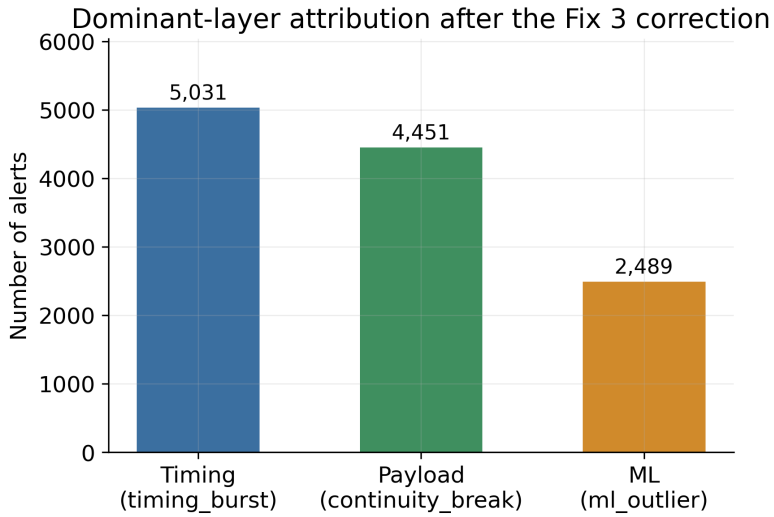

Figure 6.8: Dominant-layer attribution of the same 11,971 alerts, after the Fix 3 correction.

Figure 6.9 shows the fusion-score distribution of the first 1,000 alerts recorded by the corrected pipeline, against the three severity thresholds from Appendix B. The distribution is multi-modal rather than a single spike, and no bar sits at the top of the 0–2 range used internally by the fusion score: this is the direct, visual counterpart of the 0% ML-score-saturation figure in Table 6.4 — before the correction, the great majority of scores were pinned at the top of this range instead of spreading across it.

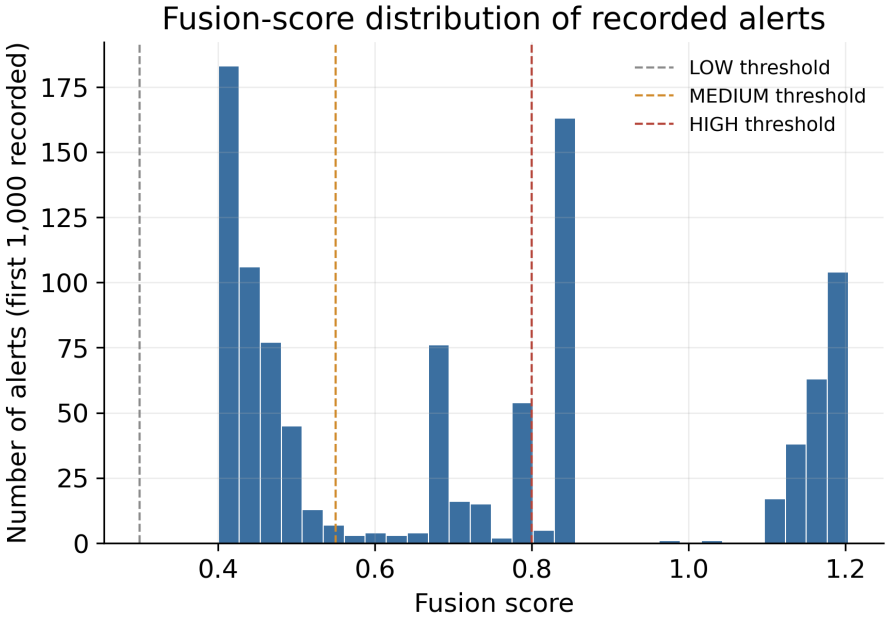

Figure 6.9: Fusion-score distribution of the first 1,000 alerts recorded by the corrected pipeline, against the LOW/MEDIUM/HIGH severity thresholds.

Figure 5.4 (Chapter 5) already shows this live alert list and a freshly-generated explanation for the selected alert, captured on the same 97,741-frame recording used throughout this section: the selected alert's identifier, severity, and score in the context bar (`0x7EC`, LOW, 0.415) match

exactly the summary text produced by the explanation engine, with no trace of an unrelated alert.

6.11 Export validation

CSV export was verified by inspecting the exported file directly: every row's time, identifier, severity, score, and attack-type columns matched the corresponding on-screen alert, and the parsed reason column reflected the Fix 3 correction. PDF export was verified by generating a report from the live application and confirming the file opens correctly and renders the same summary statistics and alert table as the CSV export.

6.12 Summary of engineering fixes

Table 6.5 summarizes every confirmed defect addressed during this project and its verification status.

Table 6.5: Summary of engineering fixes and their validation status.

#	Defect	Status
1	Vehicle model never selected for scoring	Fixed, verified
2	Driving-state misclassified as "parking"	Fixed, verified
3	Raw diagnostic fields dominating the displayed alert cause (6 occurrences)	Fixed, verified
4	Non-CSV replay formats bypassing the canonical parser	Fixed, verified
5	Alert-polling timer left permanently stopped after an import	Fixed, verified
6	AI explanation cache serving stale, cross-session data	Fixed, verified (two-part fix)

Figure 6.10 places the six fixes on the engineering timeline that produced them: the first four came directly out of the root-cause analysis of Section 6.4, while the last two only surfaced once the corrected pipeline was re-validated end to end in the live application rather than through the backend API alone — the exact reason Pass 3 of the methodology (Table 6.1) ends with a full end-to-end re-run and not just a backend re-test.

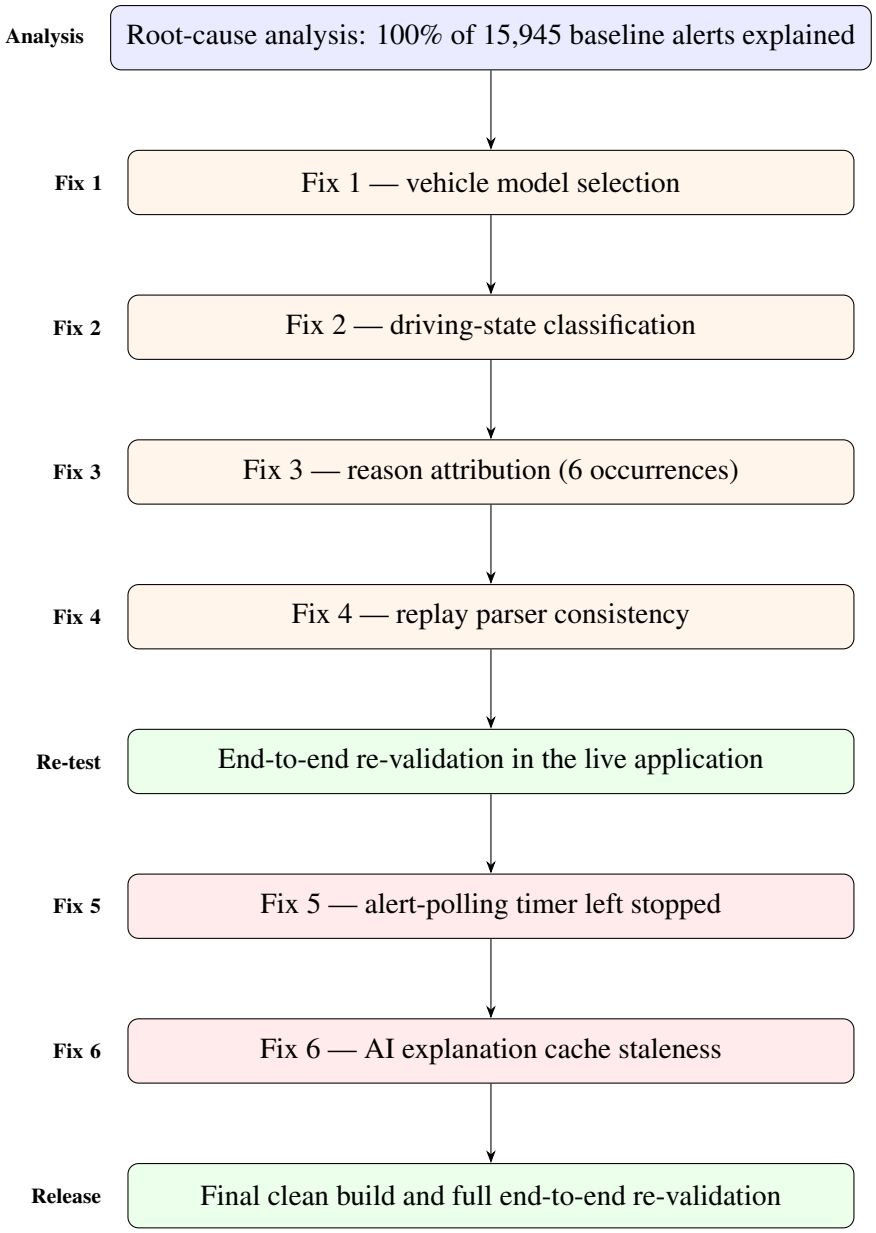

Figure 6.10: Bug-fix timeline: from root-cause analysis to the final Release Candidate.

6.13 Discussion

Every defect in Table 6.5 shares a common shape: the surrounding code did not crash and did not raise a visible error, so nothing pointed at the defect except a deliberate, evidence-based comparison between what the system reported and what was independently known to be true (the vehicle actually being driven, the file actually being a valid log, the alert actually being the one on screen). This is the practical argument, beyond the specific numbers in Table 6.4, for validating an anomaly-detection system against real, independently-understood data rather than trusting that a correctly-structured pipeline behaves correctly end to end.

6.14 Limitations

The live hardware capture path (Chapter 4) remains code-complete but unvalidated against a physical vehicle, because the ELM327 adapter needed for that test did not arrive during the project. The AI explanation's "similar historical case" feature was found, during this validation phase, to intentionally persist across sessions by design (Chapter 4); this is documented here as a known, deliberate characteristic rather than a defect, but it means an analyst comparing two unrelated vehicles may see a cross-referenced historical case, which is worth keeping in mind when reading an explanation. The CSV exporter writes the `Score` column with a French-locale decimal comma without quoting it, which shifts every later column when the file is read by a plain comma-delimited parser (Appendix D); this is a minor formatting defect in the export path itself, distinct from the six defects listed in Table 6.5, and does not affect any detection or explanation result. Finally, the root-cause analysis in Section 6.4 was performed on one recording from one vehicle; the timing- and payload-dominant shares found structurally explainable there are not guaranteed to hold in the same proportions on a different vehicle or driving pattern.

6.15 Future improvements

Three of the limitations above point to concrete future work: validating the hardware capture path once an adapter is available, closing M2 from the project planning (Chapter 3); extending the root-cause methodology used in Section 6.4 to additional vehicles, to check whether the same timing/payload/ML split holds; and scoping the historical-case feature per vehicle or per session, if cross-vehicle references turn out to be more confusing than useful to analysts in practice. The CSV formatting defect is not included here, since its fix is a self-contained, well-understood change to the export code rather than an open engineering question.

6.16 Conclusion

This chapter presented the validation methodology followed, the real one-hour Kia EV6 recording used as the primary evidence base, the full root-cause analysis of its alert volume, the four confirmed defects it led to (vehicle model selection, state classification, reason attribution, and parser consistency), two further defects found during end-to-end re-validation in the live application (alert polling and AI-explanation cache staleness), and the measured before/after impact of every correction. All measurements reported here are on the real recording described in Section 6.3, and every fix was verified with direct, reproducible evidence rather than by adjusting thresholds. The general conclusion that follows summarizes the project as a whole.

General Conclusion

This report presented CANvisionNative, a hybrid-architecture intrusion detection system for vehicle CAN bus networks, from its motivation through its design, implementation, and validation.

Achievements

The project delivered a working desktop client and backend pipeline supporting three input modes (replay, live simulation, and live hardware capture through a shared code path), a four-layer detection pipeline (timing, payload, machine learning, temporal persistence) fused into a single score, a vehicle-specific model store trained on more than seven million CAN frames across sixteen vehicle- and dataset-specific sources, and a natural-language explanation layer combining rule-based reasoning with a Large Language Model provider chain that degrades gracefully when no external model is reachable. Beyond the software itself, the project delivered a rigorous, evidence-based validation of that software against a real one-hour vehicle recording, which is documented in full in Chapter 6.

Objectives reached

Every objective listed in Chapter 1 was reached: the four-layer fusion pipeline was built and validated; recorded logs, live simulation, and live hardware capture are all supported through the same detection engine; vehicle-specific models are trained, selected at run time, and the selection was proven correct with direct evidence after a defect was found and fixed; the desktop client supports import, replay, review, and export; the explanation layer produces a plain-language description and recommendation for every alert; and the complete pipeline was validated end to end against real data, with every defect found fixed and re-verified rather than worked around.

Scientific contributions

Chapter 2 positioned this project against the reviewed literature on three points where most anomaly-based CAN IDS designs stop short: combining several independent detection signals into one fusion score instead of relying on a single signal, selecting a genuinely per-vehicle trained model at run time instead of one generic model, and producing a human-readable explanation of a detected anomaly instead of a bare numeric score. The root-cause methodology

used in Chapter 6 — explaining 100% of the alerts produced on a real recording by their dominant contributing layer before deciding whether any correction was needed — is itself a contribution that could be reused to audit other anomaly-detection systems built on the same fusion-score design.

Engineering contributions

Six confirmed software defects were found through evidence-based testing and corrected: incorrect vehicle-model selection, driving-state misclassification, mislabeled root-cause attribution (found duplicated across four files), a replay-parser inconsistency across log formats, a permanently-stopped alert-polling timer, and a stale-data leak in the AI explanation cache. Each was diagnosed to its exact root cause before being changed, and each fix was verified with direct, reproducible evidence — measured on the same real recording before and after — rather than by tuning detection thresholds. This produced a measured 25% reduction in alert volume and eliminated machine-learning score saturation entirely, without artificially suppressing any alert.

Limitations

As detailed in Chapter 6, the live hardware capture path remains unvalidated on a physical vehicle, pending the arrival of a compatible adapter, and the root-cause proportions found on the Kia EV6 recording are not guaranteed to generalize to every vehicle without further testing.

Future work

The immediate next step is the hardware validation described in Chapter 3 as milestone M2, followed by extending the evidence-based root-cause methodology of Chapter 6 to additional vehicle profiles, and refining the scope of the historical-case reference feature discussed in Section 6.9.2.

Overall, this project shows that a desktop-class, multi-layer, explainable intrusion detection system for vehicle CAN bus networks can be built with commodity hardware and open datasets, and — just as importantly — that it can be held to the same standard of evidence as any other safety-relevant piece of software before it is trusted.

Bibliography

[1] International Organization for Standardization, *ISO 11898-1:2015 — road vehicles — controller area network (can) — part 1: data link layer and physical signalling*, ISO Standard, 2015.

[2] International Organization for Standardization, *ISO 14229-1:2020 — road vehicles — unified diagnostic services (uds) — part 1: application layer*, ISO Standard, 2020.

[3] International Organization for Standardization, *ISO 15765-2:2016 — road vehicles — diagnostic communication over controller area network (docan) — part 2: transport protocol and network layer services*, ISO Standard, 2016.

[4] SAE International, *SAE J1979 — e/e diagnostic test modes*, SAE Standard, 2017.

[5] K. Koscher et al., "Experimental security analysis of a modern automobile", in *Proceedings of the IEEE Symposium on Security and Privacy*, IEEE, 2010, pp. 447–462.

[6] C. Miller and C. Valasek, *Remote exploitation of an unaltered passenger vehicle*, Technical white paper, presented at Black Hat USA, 2015.

[7] M. E. Verma, M. D. Iannacone, R. A. Bridges, S. C. Hollifield, B. Kay, and F. L. Combs, "Road: the real ornl automotive dynamometer controller area network intrusion detection dataset (with a comprehensive can ids dataset survey)", *arXiv preprint arXiv:2012.14600*, 2020.

[8] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest", in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, IEEE, 2008, pp. 413–422.

[9] J. MacQueen, "Some methods for classification and analysis of multivariate observations", *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, no. 14, pp. 281–297, 1967.

[10] E. Seo, H. M. Song, and H. K. Kim, "GIDS: GAN based intrusion detection system for in-vehicle network", in *Proceedings of the 16th Annual Conference on Privacy, Security and Trust (PST)*, IEEE, 2018.

[11] S. Ramírez, *FastAPI: modern, fast (high-performance) web framework for building APIs with Python*, https://fastapi.tiangolo.com, Accessed 2026, 2023.

[12] ASAM e.V., *ASAM MDF — measurement data format, version 4*, ASAM Technical Standard, 2021.

[13] F. Pedregosa et al., "Scikit-learn: machine learning in Python", *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

Appendix A

REST API Endpoints

Table A.1 lists the REST endpoints exposed by `backend/api/server.py`, grouped by function. All endpoints are served on `http://127.0.0.1:8765` and consumed by the desktop client only.

Table A.1: Backend REST API endpoints.

Method	Path	Purpose
GET	`/health`	Backend health check
GET	`/vehicle`	Synthetic / live vehicle snapshot
GET	`/metrics`	Runtime metrics
GET	`/api/db/stats`	SQLite database statistics (frame count, DB size, recent alerts)
POST	`/analyze`	Offline batch analysis of one or more uploaded log files
GET	`/incidents`	Grouped incidents from the last analysis
POST	`/inject`	Attack injection into an uploaded log
POST	`/feedback`	Analyst feedback on a frame label
POST	`/infer`	One-off inference on posted features
POST	`/api/validation/score_file`	Batch validation scoring
POST	`/api/live/start`	Start live simulated session
POST	`/api/live/stop`	Stop live session
GET	`/api/live/signals`	Recent raw live frames
GET	`/api/live/decoded-signals`	Recent decoded live signals
GET	`/api/live/alerts`	Live alert buffer
GET	`/api/explain/{alert_index}`	AI explanation for an alert
GET	`/api/live/status`	Live/simulator status
GET	`/api/runtime/statistics`	Runtime IDS statistics
GET	`/api/runtime/health`	Runtime engine health
POST	`/api/replay/start`	Start a replay subprocess
POST	`/api/replay/stop`	Stop the running replay

Method	Path	Purpose
GET	`/api/replay/status`	Replay progress / completion state
GET	`/api/replay/results`	Latest replay result file
GET	`/api/replay/alerts`	Alerts from the latest replay summary
GET	`/api/replay/partial-alerts`	Partial alerts while replay is running
POST	`/api/simulator/generate`	Generate a simulated attack
GET	`/api/simulator/attacks`	List available simulated attack types
GET	`/api/validation/reports`	List validation report files
GET	`/api/vehicles`	Supported vehicle profile list
GET	`/api/vehicles/{vehicle_id}/can_ids`	CAN identifiers defined in a vehicle's DBC
POST	`/api/offline/analyze`	Start offline batch analysis
GET	`/api/offline/status/{session_id}`	Offline analysis progress
GET	`/api/offline/frames/{session_id}`	Paginated decoded frames
GET	`/api/offline/summary/{session_id}`	Offline analysis summary
POST	`/api/mf4/convert`	Convert an MF4 file to CSV
POST	`/api/live/hardware/connect`	Connect a physical CAN adapter
POST	`/api/live/hardware/disconnect`	Disconnect the adapter
GET	`/api/live/hardware/status`	Hardware connection status
GET	`/api/live/hardware/ports`	Available serial ports
GET	`/api/live/hardware/check`	Hardware availability check
GET	`/api/system/info`	System / build information
GET	`/api/settings`	Current settings
POST	`/api/settings`	Update settings
POST	`/api/chat`	Conversational Q&A about an alert
POST	`/api/session/record/start`	Start session recording
POST	`/api/session/record/stop`	Stop session recording
GET	`/api/session/record/status`	Recording status
GET	`/api/session/list`	List recorded sessions
GET	`/api/system/health`	Aggregated system health

Appendix B

Configuration

B.1 Requirements

Table B.1 lists the software requirements to build and run CANvisionNative, as documented in the project's own setup guide. Table B.2 lists the hardware requirements; only the last row is needed for the live hardware capture mode (Section 4.4) — every other feature, including the full validation in Chapter 6, runs on the desktop machine alone.

Table B.1: Software requirements.

Component	**Version**
Windows	10 or 11, 64-bit
.NET Runtime	4.8
Python	3.11 – 3.14
Git	Any (for cloning the repository)

Table B.2: Hardware requirements.

Component	Requirement	Notes
Desktop / laptop	Any machine able to run Windows 10/11 and Python 3.11+	Used for every mode (replay, simulation, hardware capture)
Disk space	A few GB free	Trained model store, DBC files, and recorded logs
USB port	1 free port	Only for the ELM327 / USB-CAN adapter below
CAN adapter	ELM327 USB (slcan), PEAK PCAN-USB, or a SocketCAN-compatible interface	Required only for live hardware capture (FR9); not

B.2 Key Python dependencies

- `fastapi`, `uvicorn` — REST API server
- `scikit-learn`, `joblib` — Isolation Forest and K-Means pipeline
- `pandas`, `numpy` — feature engineering
- `asammdf` — MF4 decoding
- `cantools` — DBC-based signal decoding
- `python-can` — hardware CAN adapter access

B.3 Backend startup

```
python -m uvicorn backend.api.server:app --host 127.0.0.1 --port 8765
```

The desktop client automatically starts and stops this process when a Python interpreter is found at the expected path, and exposes the same functionality if the backend is instead started manually beforehand.

B.4 Persisted configuration

Client-side settings (alert threshold, LLM provider key, hardware interface parameters) are persisted to `%AppData%\CANvision\config.json` and reloaded on startup. The LLM provider key, if configured, is applied as an environment variable so the backend can pick it up without a separate configuration step.

B.5 Fusion score configuration

Table B.3: Default fusion and severity configuration.

Parameter	**Value**
Fusion score range	0.0 – 2.0
Layer weights	timing 0.35, payload 0.35, ML 0.20, persistence 0.10
Severity thresholds	LOW $\geq$ 0.30, MEDIUM $\geq$ 0.55, HIGH $\geq$ 0.80, CRITICAL $\geq$ 1.00
Backend port	8765

Appendix C

Datasets

C.1 Training corpus

The machine-learning models used by CANvisionNative were trained on a corpus of **more than seven million CAN frames** drawn from sixty-six sources, summarized in Table C.1.

Table C.1: Main sources of the training corpus.

Source	Description	Frames
ROAD ambient [7]	University-collected ambient drives (12 sessions)	1,737,195
Car-Hacking (normal rows) [10]	Public benchmark, normal-traffic subset	1,200,000
Nissan Leaf	Community EV CAN logs	1,163,181
BMW i3	Community EV CAN logs	1,044,993
Kia EV6	Community EV CAN logs	886,368
13 additional sources	Kia Soul, Jaguar I-PACE, MG ZS EV, Tesla, others	∼1,032,329

This corpus produced sixteen vehicle- or dataset-specific model folders (including `kia_ev6`, `nissan_leaf`, `bmw_i3`, `skoda_enyaq`, `tesla`, and others) plus one generic fallback bundle (`general`), stored under `assets/models/`, alongside the shared `kmeans_state_model.joblib` and `state_scaler.joblib` used by the state classifier (Section 5.5.3).

C.2 Primary validation dataset

The main validation dataset used in Chapter 6 is a real, one-hour Kia EV6 recording from the CSS Electronics EV Data Pack (`00000158-64CBC290.MF4`), captured by a CANedge logger performing active UDS polling. After MF4 conversion, the recording contains **97,741 frames**, decoded with the Hyundai/Kia UDS DBC file (`can1-hyundai-kia-uds-v2.4.dbc`).

C.3 Cross-validation dataset

To test for cross-session and cross-dataset state leakage in the AI explanation module (Section 6.9.2), a second, independent real recording was used: a Renault Clio DoS-attack candump log, from a different vehicle and a different capture tool, containing frames on identifiers unrelated to the Kia EV6 recording (for example `0xC6`, `0x511`).

Appendix D

Additional Validation Data

D.1 Post-correction alert breakdown

Table D.1 and Table D.2 give the full severity and reason-attribution breakdown of the 11,971 alerts produced by the corrected pipeline on the Kia EV6 recording (Chapter 6), superseding the pre-correction breakdown of Table 6.3.

Table D.1: Severity distribution after correction.

Severity	**Count**	**Share**
LOW	4,748	39.7%
MEDIUM	2,069	17.3%
HIGH	2,279	19.0%
CRITICAL	2,875	24.0%
Total	**11,971**	**100%**

Table D.2: Reason-attribution distribution after the Fix 3 correction.

Dominant layer (bucket)	**Count**	**Share**
Timing (`timing_burst`)	5,031	42.0%
Payload (`continuity_break`)	4,451	37.2%
ML (`ml_outlier`)	2,489	20.8%
Total	**11,971**	**100%**

D.2 Top alerting CAN identifiers

Figure D.1 ranks the eight most frequent identifiers in the post-correction alert list. The identifiers `0x7EC`, `0x7E4`, `0x7BB`, `0x7A8`, `0x7B3`, and `0x7A0` all fall in the `0x7A0–0x7EC` range

reserved for UDS diagnostic request/response pairs, consistent with the recording's active-polling capture method described in Appendix C. Identifier `0x19`, the single most frequent alert source, and `0x13` are broadcast identifiers rather than diagnostic ones, and correspond to the high-frequency battery and drivetrain messages discussed in Section 6.4.

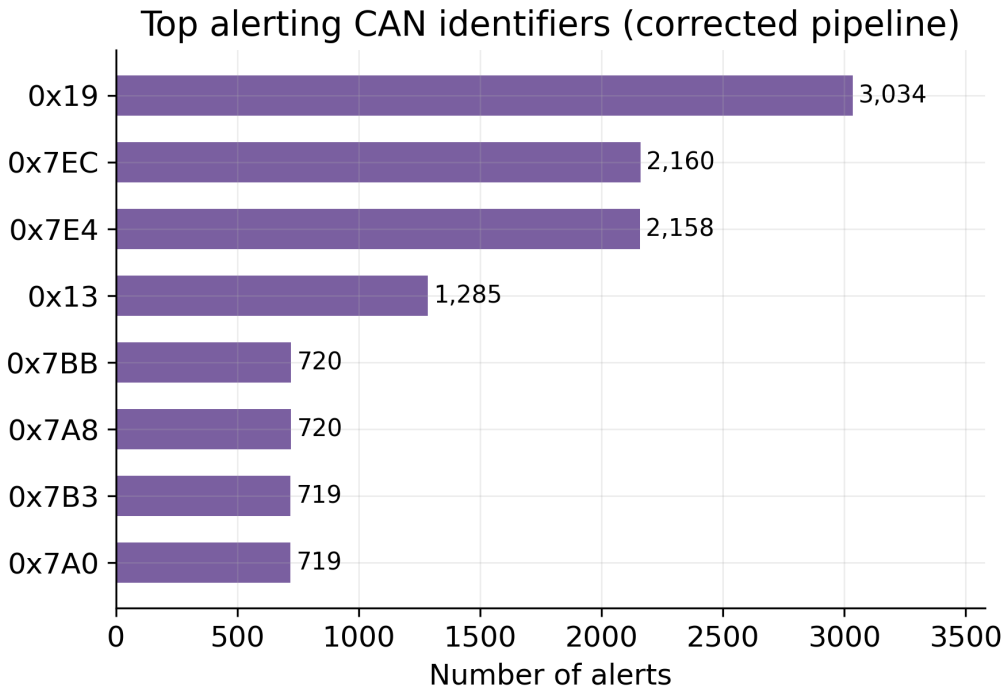

Figure D.1: Top alerting CAN identifiers in the corrected pipeline's alert list.

D.3 Additional screenshots

Figure D.2 shows the Dashboard view during a live session: the fusion score gauge, active-threat panel, and IDS quick statistics.

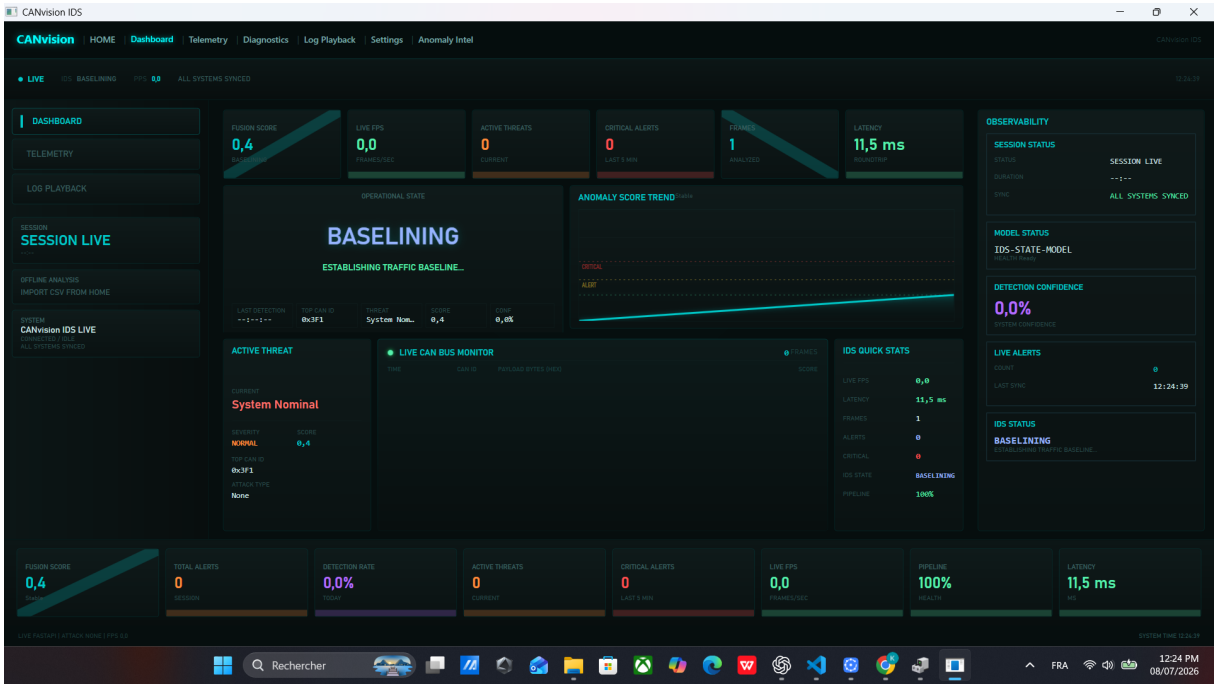

Figure D.2: Dashboard view: fusion score, active-threat panel, and IDS quick statistics.

Figure D.3 shows the Settings view, including the hardware interface fields, the alert threshold slider, and the vehicle profile list also summarized in Table 5.2.

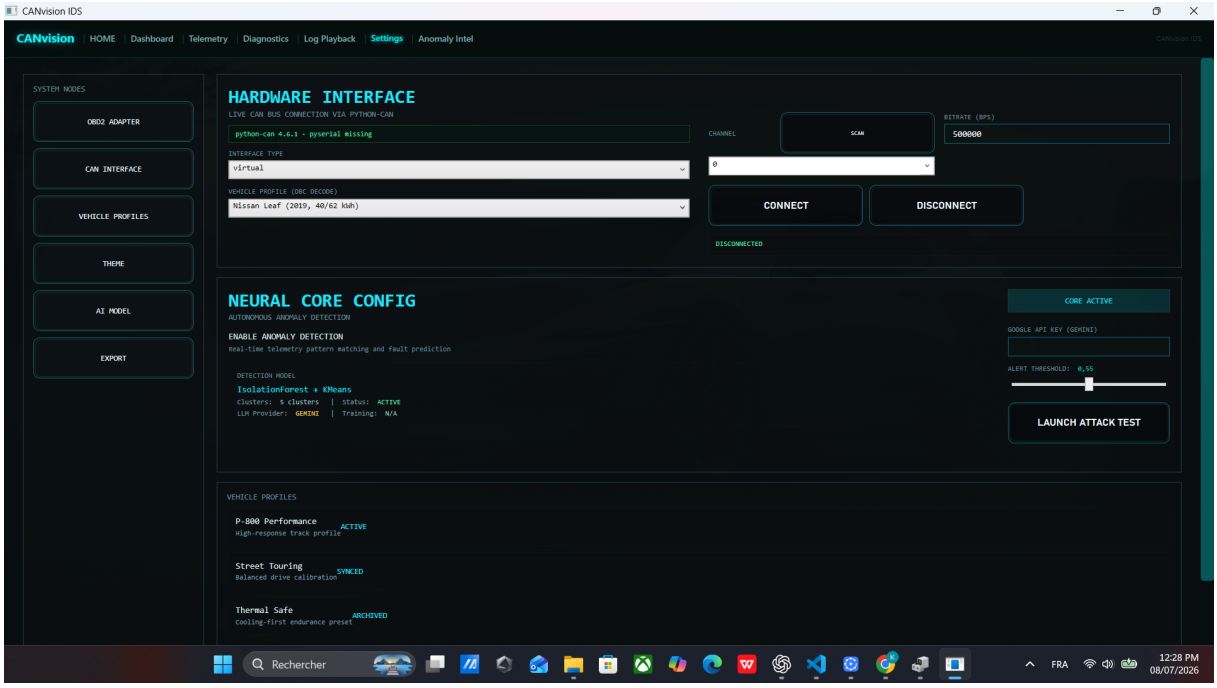

Figure D.3: Settings view: hardware interface, neural core configuration, and vehicle profiles.

Figure D.4 shows a rendering of the real exported CSV file, reconstructed from the raw export because the exporter writes the `Score` column with a French-locale decimal comma (for example `0,415`) without quoting it, which shifts every later column when the file is opened with a plain, comma-delimited reader. This is a genuine, minor formatting quirk of the export

path, noted here for completeness; it does not affect any of the validation results in Chapter 6, all of which were read from the underlying JSON summary rather than the CSV export.

Exported alerts_20260708_125731.csv — first 12 rows (real export, corrected pipeline)

Time	CAN_ID	Severity	AttackType	Score
01:00:00.028	0x7EC	LOW	CONTINUITY_BREAK	0.415
01:00:00.517	0x19	LOW	CONTINUITY_BREAK	0.440
01:00:00.521	0x7BB	MEDIUM	CONTINUITY_BREAK	0.624
01:00:01.000	0x7E4	LOW	CONTINUITY_BREAK	0.493
01:00:01.028	0x7EC	MEDIUM	CONTINUITY_BREAK	0.607
01:00:01.528	0x19	LOW	CONTINUITY_BREAK	0.426
01:00:01.530	0x7A8	LOW	CONTINUITY_BREAK	0.414
01:00:02.000	0x7E4	MEDIUM	CONTINUITY_BREAK	0.636
01:00:02.028	0x7EC	LOW	CONTINUITY_BREAK	0.541
01:00:02.540	0x19	LOW	CONTINUITY_BREAK	0.428
01:00:04.573	0x19	LOW	CONTINUITY_BREAK	0.486
01:00:05.000	0x7E4	HIGH	TIMING_BURST	0.838

Figure D.4: Exported CSV file (reconstructed preview), Time/CAN_ID/Severity/AttackType/Score columns.

Figure D.5 shows the first page of the exported PDF report, generated from the same completed replay: header, summary statistics, and the start of the paginated alert table.

CANVISION IDS

INTRUSION DETECTION SYSTEM — ALERT REPORT

Generated: 2026-07-08 13:21:13

ANALYSIS SUMMARY

TOTAL ALERTS	CRITICAL	HIGH / WARNING	LOW
500	103	78	319

ALERT DETAILS

#	TIME	CAN-ID	SEVERITY	ATTACK TYPE	SCORE	REASON
1	01:00:00.028	0x7EC	LOW	CONTINUITY_BREAK	0,415	timing=0.000, payload=1.185, ml=0.000, persistence=0.000, freq=36Hz, can_id_entropy=0.000
2	01:00:00.517	0x19	LOW	CONTINUITY_BREAK	0,440	timing=0.004, payload=0.915, ml=0.591, persistence=0.000, freq=10Hz, can_id_entropy=1.535
3	01:00:00.521	0x7BB	MEDIUM	CONTINUITY_BREAK	0,624	timing=0.014, payload=1.769, ml=0.000, persistence=0.000, freq=96Hz, can_id_entropy=1.535
4	01:00:01.000	0x7E4	LOW	CONTINUITY_BREAK	0,493	timing=0.470, payload=0.938, ml=0.000, persistence=0.000, freq=2Hz, can_id_entropy=1.535
5	01:00:01.028	0x7EC	MEDIUM	CONTINUITY_BREAK	0,607	timing=0.266, payload=1.045, ml=0.740, persistence=0.000, freq=10Hz, can_id_entropy=2.163
6	01:00:01.528	0x19	LOW	CONTINUITY_BREAK	0,426	timing=0.004, payload=0.794, ml=0.733, persistence=0.000, freq=10Hz, can_id_entropy=2.262
7	01:00:01.530	0x7A8	LOW	CONTINUITY_BREAK	0,414	timing=0.000, payload=1.183, ml=0.000, persistence=0.000, freq=42Hz, can_id_entropy=2.262
8	01:00:02.000	0x7E4	MEDIUM	CONTINUITY_BREAK	0,636	timing=0.470, payload=0.924, ml=0.740, persistence=0.000, freq=2Hz, can_id_entropy=2.262
9	01:00:02.028	0x7EC	LOW	CONTINUITY_BREAK	0,541	timing=0.289, payload=0.836, ml=0.739, persistence=0.000, freq=8Hz, can_id_entropy=2.262
10	01:00:02.540	0x19	LOW	CONTINUITY_BREAK	0,428	timing=0.003, payload=0.801, ml=0.733, persistence=0.000, freq=10Hz, can_id_entropy=2.281
11	01:00:04.573	0x19	LOW	CONTINUITY_BREAK	0,486	timing=0.003, payload=0.967, ml=0.733, persistence=0.000, freq=10Hz, can_id_entropy=2.267
12	01:00:05.000	0x7E4	HIGH	TIMING_BURST	0,838	timing=1.044, payload=0.925, ml=0.740, persistence=0.010, freq=1Hz, can_id_entropy=2.267
13	01:00:05.001	0x7EC	MEDIUM	CONTINUITY_BREAK	0,753	timing=0.644, payload=1.080, ml=0.742, persistence=0.010, freq=4Hz, can_id_entropy=2.267
14	01:00:05.481	0x13	LOW	ML_OUTLIER	0,478	timing=0.433, payload=0.505, ml=0.737, persistence=0.020, freq=1Hz, can_id_entropy=2.151
15	01:00:05.482	0x15	LOW	ML_OUTLIER	0,457	timing=0.433, payload=0.450, ml=0.732, persistence=0.020, freq=1Hz, can_id_entropy=2.151
16	01:00:05.490	0x7BB	CRITICAL	TIMING_BURST	1,194	timing=2.000, payload=0.981, ml=0.740, persistence=0.030, freq=1Hz, can_id_entropy=2.151
17	01:00:05.500	0x7B3	HIGH	TIMING_BURST	0,978	timing=2.000, payload=0.784, ml=0.000, persistence=0.040, freq=0Hz, can_id_entropy=2.151
18	01:00:05.689	0x19	MEDIUM	CONTINUITY_BREAK	0,557	timing=0.003, payload=1.141, ml=0.733, persistence=0.100, freq=10Hz, can_id_entropy=2.235
19	01:00:06.000	0x5	LOW	ML_OUTLIER	0,404	timing=0.400, payload=0.312, ml=0.727, persistence=0.100, freq=1Hz, can_id_entropy=2.235
20	01:00:06.000	0x7E4	HIGH	TIMING_BURST	0,818	timing=0.945, payload=0.938, ml=0.740, persistence=0.110, freq=1Hz, can_id_entropy=2.235
21	01:00:06.028	0x7EC	MEDIUM	CONTINUITY_BREAK	0,720	timing=0.557, payload=1.045, ml=0.742, persistence=0.110, freq=5Hz, can_id_entropy=2.235
22	01:00:06.483	0x13	LOW	ML_OUTLIER	0,484	timing=0.400, payload=0.531, ml=0.737, persistence=0.110, freq=1Hz, can_id_entropy=2.175
23	01:00:06.500	0x7A0	CRITICAL	TIMING_BURST	1,017	timing=2.000, payload=0.873, ml=0.000, persistence=0.120, freq=0Hz, can_id_entropy=2.175
24	01:00:06.504	0x7A8	CRITICAL	TIMING_BURST	1,171	timing=1.836, payload=1.048, ml=0.742, persistence=0.130, freq=1Hz, can_id_entropy=2.175
25	01:00:07.000	0x7E4	MEDIUM	CONTINUITY_BREAK	0,797	timing=0.877, payload=0.924, ml=0.737, persistence=0.190, freq=1Hz, can_id_entropy=2.254
26	01:00:07.000	0x5	LOW	ML_OUTLIER	0,404	timing=0.373, payload=0.312, ml=0.727, persistence=0.190, freq=1Hz, can_id_entropy=2.254
27	01:00:07.028	0x7EC	MEDIUM	CONTINUITY_BREAK	0,643	timing=0.523, payload=0.836, ml=0.740, persistence=0.190, freq=6Hz, can_id_entropy=2.254
28	01:00:07.115	0x19	LOW	CONTINUITY_BREAK	0,522	timing=0.003, payload=1.016, ml=0.733, persistence=0.190, freq=10Hz, can_id_entropy=2.254
29	01:00:07.496	0x13	LOW	ML_OUTLIER	0,466	timing=0.371, payload=0.485, ml=0.737, persistence=0.190, freq=1Hz, can_id_entropy=2.254
30	01:00:08.230	0x19	LOW	ML_OUTLIER	0,413	timing=0.003, payload=0.705, ml=0.733, persistence=0.180, freq=10Hz, can_id_entropy=2.268
31	01:00:09.000	0x5	LOW	ML_OUTLIER	0,409	timing=0.331, payload=0.375, ml=0.727, persistence=0.170, freq=1Hz, can_id_entropy=2.227
32	01:00:09.356	0x19	LOW	CONTINUITY_BREAK	0,439	timing=0.003, payload=0.785, ml=0.733, persistence=0.170, freq=10Hz, can_id_entropy=2.227
33	01:00:09.480	0x13	LOW	ML_OUTLIER	0,476	timing=0.330, payload=0.567, ml=0.737, persistence=0.140, freq=1Hz, can_id_entropy=2.227

CANvision Native — CAN Bus Intrusion Detection System

Page 1

Figure D.5: Exported PDF report, first page.